

Agentic AI Mastery Handbook

Chapter 1 – Agentic AI Fundamentals

Part 1 – Agentic AI, AI Agents & Agent Architecture

Interview Importance: ★ ★ ★ ★ ★

Difficulty: Beginner

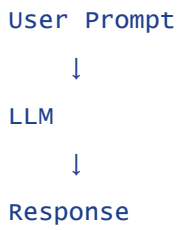
Prerequisites: LLM Fundamentals, Prompt Engineering, Basic GenAI

1. What is Agentic AI?

★ ★ ★ ★ ★

Agentic AI refers to AI systems that can work toward a goal by making decisions, taking actions using tools, observing the results, and deciding what to do next.

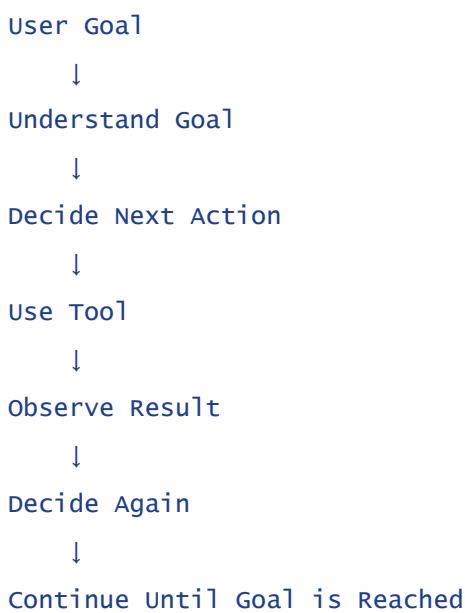
A normal LLM interaction is usually:



```
graph TD; A[User Prompt] --> B[LLM]; B --> C[Response];
```

User Prompt
↓
LLM
↓
Response

An agentic system can instead work like:



```
graph TD; A[User Goal] --> B[Understand Goal]; B --> C[Decide Next Action]; C --> D[Use Tool]; D --> E[Observe Result]; E --> F[Decide Again]; F --> G[Continue Until Goal is Reached];
```

User Goal
↓
Understand Goal
↓
Decide Next Action
↓
Use Tool
↓
Observe Result
↓
Decide Again
↓
Continue Until Goal is Reached

The important difference is:

A normal LLM primarily **generates**, while an AI agent can **decide and act** as part of a workflow.

2. Simple Example

Suppose the user asks:

"What is the current weather in Delhi?"

A normal LLM may answer using its existing knowledge, but that knowledge may not contain the current weather.

An agent can:

```
graph TD; A[User asks current weather] --> B[LLM understands that live data is required]; B --> C[Select weather Tool]; C --> D[Call weather API]; D --> E[Receive Current Weather]; E --> F[LLM interprets result]; F --> G[Answer User];
```

So the LLM is no longer working alone.

It can interact with:

- APIs
- Databases
- Search systems
- Files
- Calculators
- Code
- Other services

3. LLM vs AI Agent

★★★★★ *Important Interview Question*

LLM

A basic LLM performs:

Prompt



LLM



Generated Response

Example:

User:

Explain photosynthesis.

LLM:

Photosynthesis is the process...

This is primarily a:

Prompt → Response

interaction.

AI Agent

An AI agent can perform:

Goal



Reason / Decide



Take Action



Observe Result



Decide Next Action



Final Answer

Example:

User:

Find the weather in Delhi and tell me whether I should carry an umbrella.

Agent:

Need current weather



Call weather Tool



Weather Tool returns:

Rain probability = 80%



Interpret result



Recommend umbrella

4. LLM vs Agent Comparison

Feature	Basic LLM	AI Agent
Generate text	✓	✓
Reason about request	✓	✓
Use external tools	Not inherently	✓
Access live data	Not inherently	✓ Through tools
Perform actions	Limited	✓
Observe tool results	✗ in basic flow	✓
Multi-step execution	Limited	✓
Maintain workflow state	Usually limited	✓
Adapt next step	Limited	✓

5. What Makes an AI System "Agentic"?

A useful agent generally has some combination of:

Goal

Reasoning / Decision Making

Planning

Tools

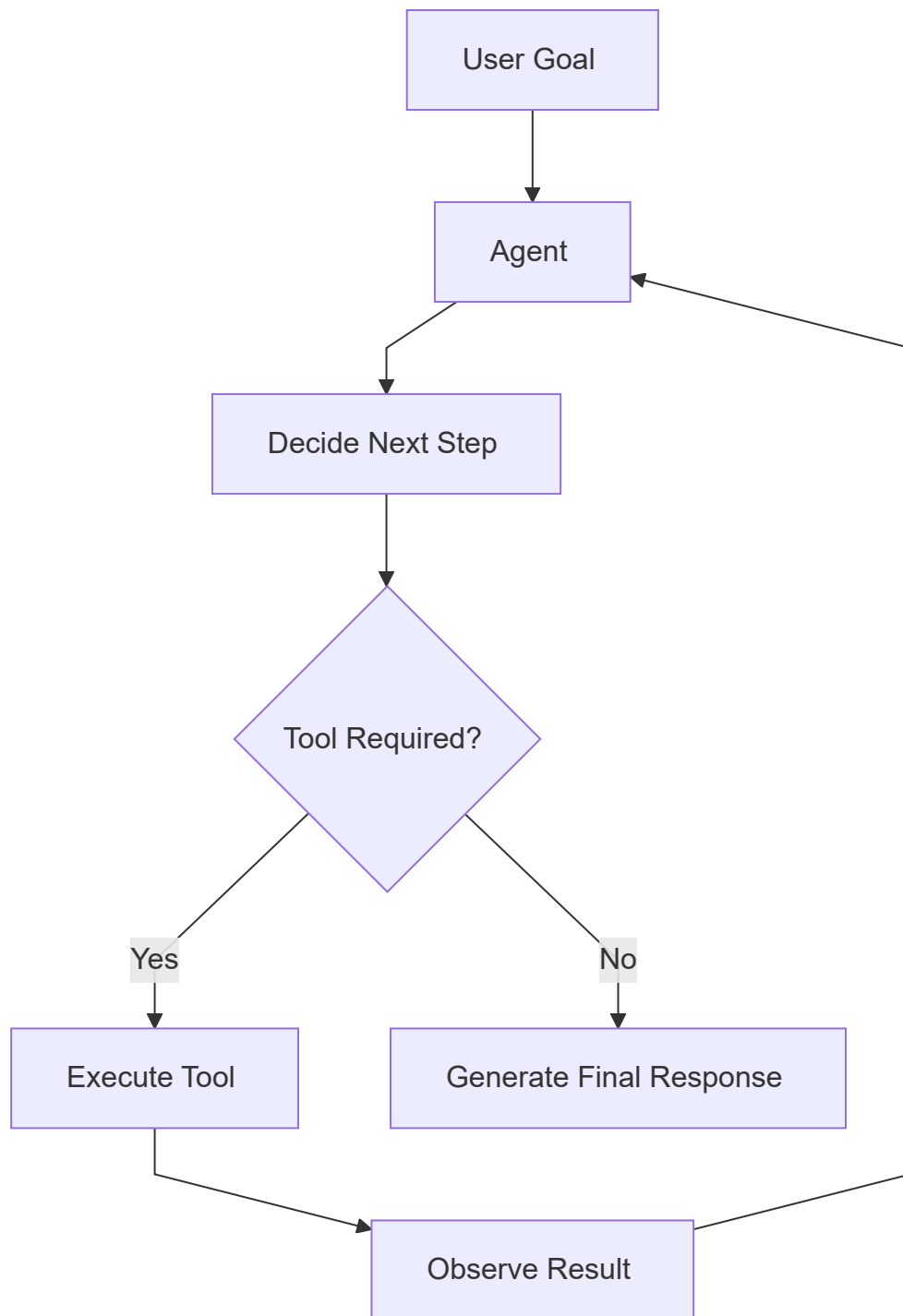
Actions

Observations

State / Memory

Control Loop

Architecture:



The important part is the loop:

```
graph TD;
    A[Decide] --> B[Act];
    B --> C[Observe];
    C --> D[Decide Again];
```

6. AI Agent vs Agentic AI

These terms are closely related but slightly different.

AI Agent

*An **AI agent** is an individual system capable of making decisions and taking actions toward a goal.*

Example:

```
graph TD;
    A[Weather Agent] --> B[User Question];
    B --> C[Agent];
    C --> D[Weather API];
    D --> E[Answer];
```

Agentic AI

Agentic AI is the broader idea of building AI systems around autonomous or semi-autonomous goal-directed behavior.

It may include:

- One Agent
- Multiple Agents
- Tools
- Memory
- Planning
- State
- Human Approval
- Complex workflows

So:

- AI Agent
=
An individual goal-directed AI system
- Agentic AI
=
Broader architecture / approach for building such systems

In everyday discussion, however, the terms are often used loosely.

7. Normal GenAI Application vs Agentic Application

Suppose we build a travel application.

Normal GenAI

User:
Plan a 3-day Delhi trip.

↓

LLM

↓

Generated Itinerary

The LLM generates a plan from the information available in its context.

Agentic AI

The system could:

User Goal

↓

Find destination information

↓

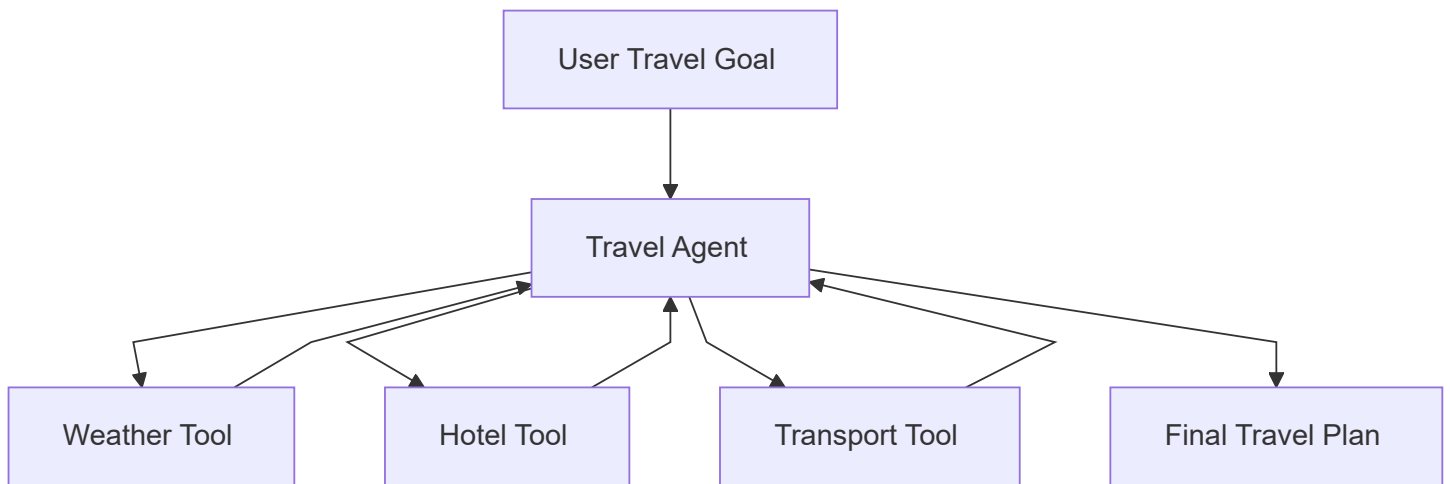
Check weather

↓

Search hotels

↓
Check travel options
↓
Compare results
↓
Build itinerary
↓
Present recommendation

Different tools may provide different capabilities.



8. Why Do We Need Agents?

LLMs are powerful, but an LLM alone has important limitations.

For example:

No guaranteed access to live information

Cannot inherently access your database

Cannot inherently send emails

Cannot inherently create bookings

Cannot inherently query private systems

Cannot inherently execute arbitrary application actions

Agents connect the LLM with external capabilities.

LLM

+

Tools

+

State

+

Control Logic

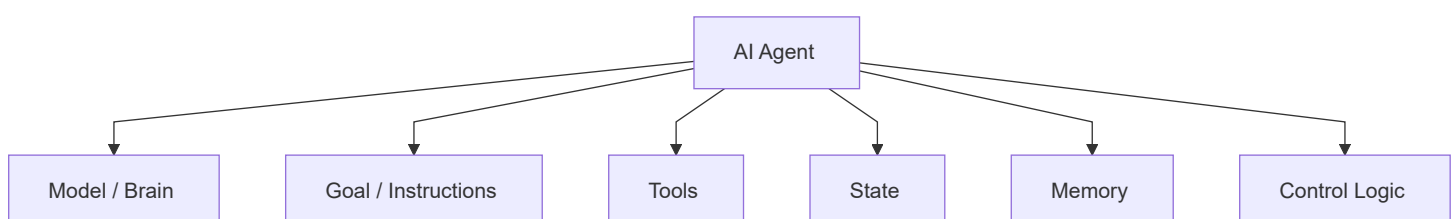
=

Agentic System

9. Core Components of an AI Agent



A practical agent can be understood using these components:



Let's understand each.

10. Component 1 – Model / Brain

The LLM acts as the main reasoning and language component.

Examples could include models from:

- OpenAI
- Gemini
- Claude
- Open-source LLMs

The model can help determine:

- What does the user want?
- Do I need a tool?
- Which tool should I use?
- What arguments should I provide?
- What should I do with the result?
- Is another action required?
- Can I answer now?

Conceptually:

Current State

+

User Goal

+

Available Tools

↓

LLM

↓

Next Decision

II. Component 2 – Goal / Instructions

An agent needs to know what it is supposed to accomplish.

Example:

You are a weather assistant.

Your goal is to provide current weather information.

Use the weather tool whenever live weather information is required.

The goal guides agent decisions.

Goal

↓

Agent Decisions

↓

Actions

Without clear instructions, an agent may:

Choose unnecessary tools

Take incorrect actions

Generate irrelevant answers

Continue unnecessarily

12. Component 3 – Tools



Tools allow an agent to interact with systems outside the LLM.

Examples:

Weather API

Search API

Calculator

Database

Email Service

Calendar

File System

Code Executor

Example tool:

```
def get_weather(city):  
    # Call weather service  
    return weather
```

The agent can decide:

```
"I need current weather."  
  
    ↓  
  
get_weather("Delhi")
```

*Tool calling will be covered properly in **Chapter 2**.*

13. LLM Knowledge vs Tools

Suppose the user asks:

```
what is photosynthesis?
```

The LLM probably does not require a tool.

```
Question  
    ↓  
LLM  
    ↓  
Answer
```

But:

```
what is the weather in Delhi right now?
```

requires live information.

Question



Agent



Weather Tool



Live Data



LLM



Answer

The agent should therefore decide:

Can I answer from available context?

OR

Do I need an external tool?

14. Component 4 – State



State represents the information required to track the current execution of an agent or workflow.

Example:

User Request

Current Step

Previous Tool Result

Messages

Intermediate Data

Execution Status

Suppose an agent needs to:

1. Get weather
2. Find suitable activities
3. Generate recommendation

After step 1, state might contain:

```
{  
  "city": "Delhi",  
  "weather": "Rainy",  
  "current_step": 2  
}
```

The system can then continue from that information.

15. Why State is Important

Without state:

Step 1

↓

Result

Step 2

↓

Previous information lost

With state:

Step 1

↓

Store Result

↓

Step 2

↓

Read Previous Result

↓

Continue

LangGraph relies heavily on this idea.

We will study state properly later.

16. Component 5 – Memory

Memory allows useful information to persist beyond the immediate step.

Examples:

Previous conversation

User preferences

Past actions

Previously discovered information

Conceptually:

Current state

→ what is happening now?

Memory

→ Useful information retained from before

State and memory are related, but they are not exactly the same.

*We will study this distinction in **Chapter 3**.*

17. Component 6 – Control Logic

Something must control:

What happens next?

Should a tool run?

Should the agent loop again?

Should execution stop?

Should a human approve an action?

This is the orchestration layer.

For a basic agent:

```
LLM
↓
Tool?
├─ Yes → Execute → Return Result → LLM
└─ No  → Final Answer
```

For complex agents, frameworks such as LangGraph can make this control flow explicit.

18. Action and Observation



Two important agent concepts are:

Action

Something the agent chooses to do.

Example:

```
Call weather API
```

Observation

The result produced by that action.

Example:

Temperature = 31°C
Rain Probability = 80%

Then:

Action
↓
External System
↓
Observation
↓
Agent

19. Agent Loop

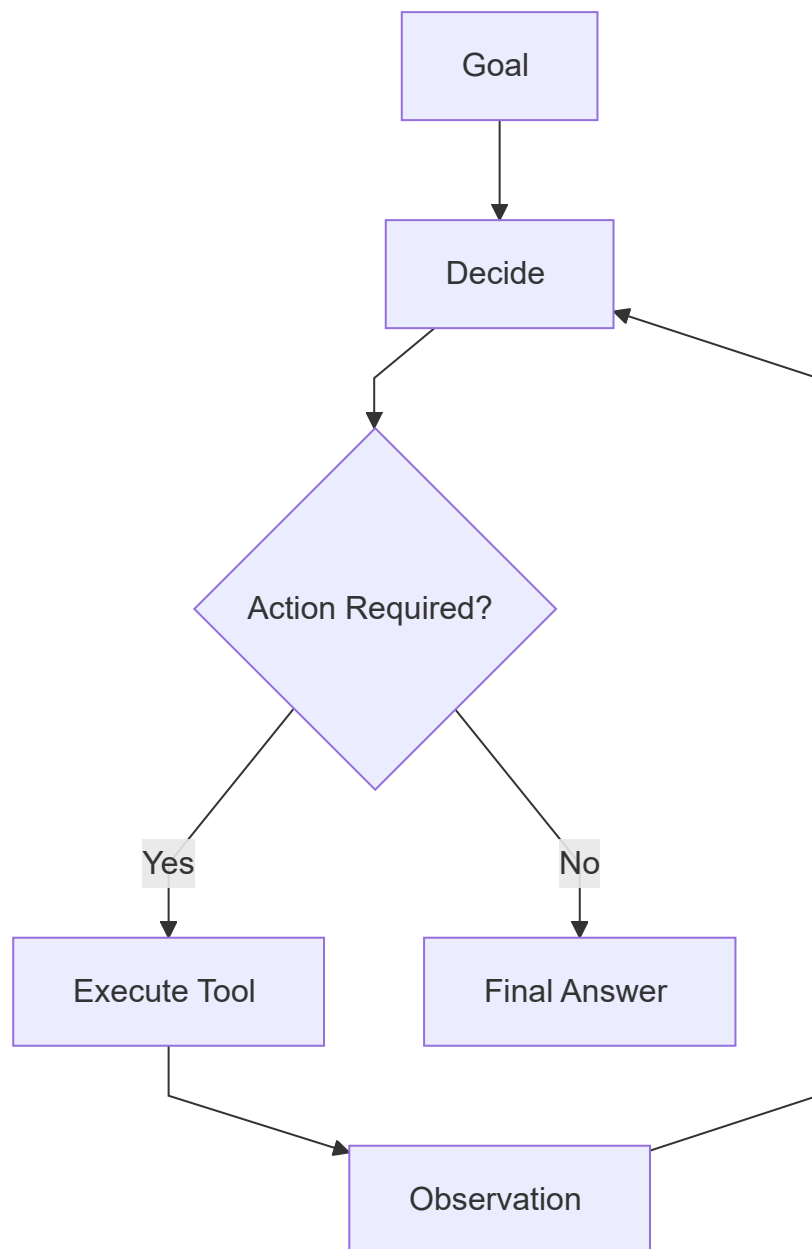


The most important mental model in Agentic AI is:

Goal
↓
Reason / Decide
↓
Action
↓
Observation

↓
Reason / Decide
↓
Action
↓
Observation
↓
...
↓
Final Answer

Diagram:



This is the core of many agent architectures.

20. Example of an Agent Loop

User asks:

"What is 15% of the current Bitcoin price?"

The agent may need two tools.

Step 1

Goal:
Calculate 15% of current Bitcoin price.

Agent decides:

I first need the current Bitcoin price.

Action:

`get_crypto_price("BTC")`

Observation:

₹X

Step 2

Agent now decides:

```
I need to calculate 15% of ₹X.
```

Action:

```
calculator("X * 0.15")
```

Observation:

```
₹Y
```

Step 3

Agent now has enough information.

```
Final Answer:  
15% of the current Bitcoin price is approximately ₹Y.
```

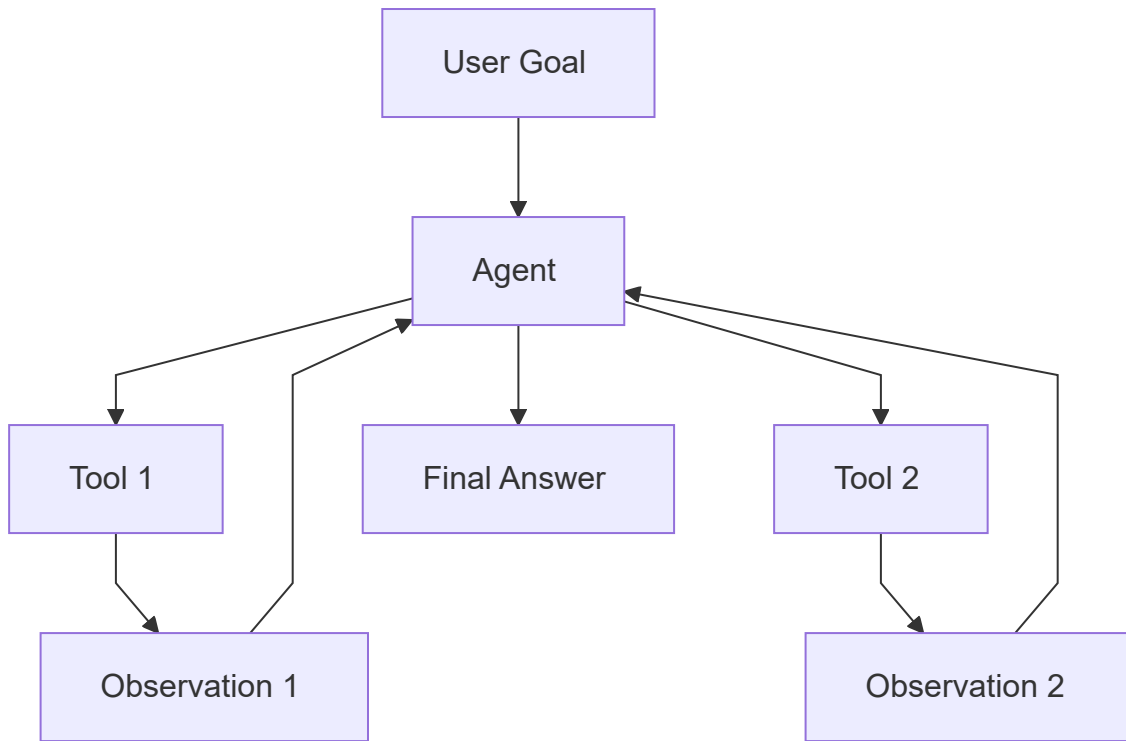
Notice that the second action depended on the first observation.

That is what makes agent workflows powerful.

21. Multi-Step Agent

A normal tool-calling application might perform one predefined action.

An agent can dynamically perform several actions.



The exact path may depend on the observations.

22. Reasoning and Planning

Agents may need to break larger goals into smaller tasks.

Example:

User Goal:

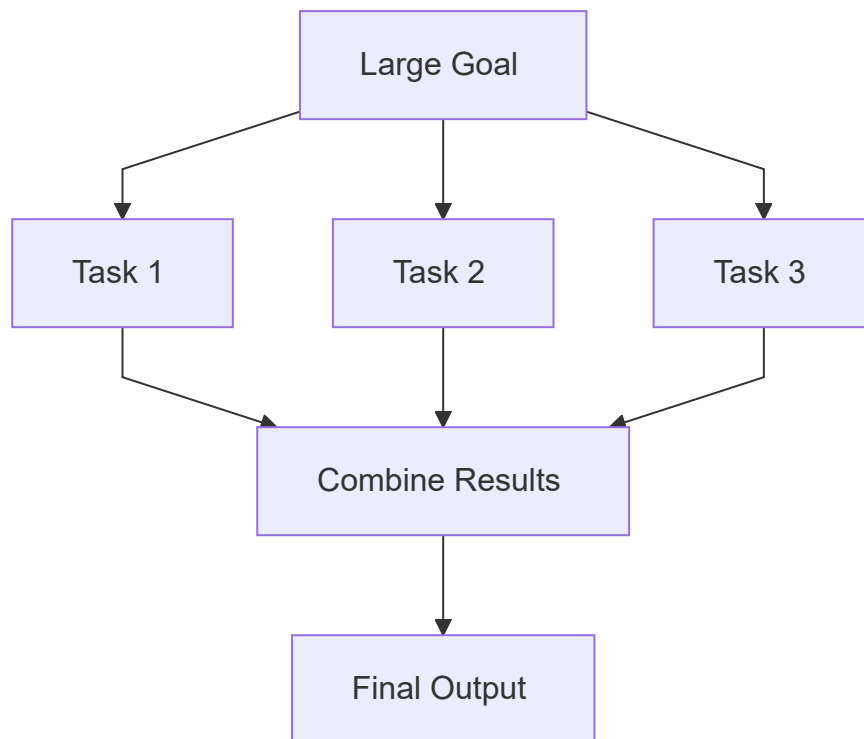
Research a company and prepare a short report.

Possible plan:

1. Find company information
2. Find products
3. Find recent information
4. Organize findings
5. Generate report

This is:

Task Decomposition



We will study planning properly in Chapter 3.

23. What is ReAct?



ReAct stands for:

Reasoning + Acting

It is an influential pattern where an agent alternates between reasoning about what to do and taking actions in an environment.

Conceptually:

```
Reason
↓
Action
↓
Observation
↓
Reason
↓
Action
↓
Observation
↓
Final Answer
```

Example:

```
User:
What is the weather in Delhi and should
I carry an umbrella?
```

Reason:

I need live weather information.

Action:

Call weather tool.

Observation:

Heavy rain expected.

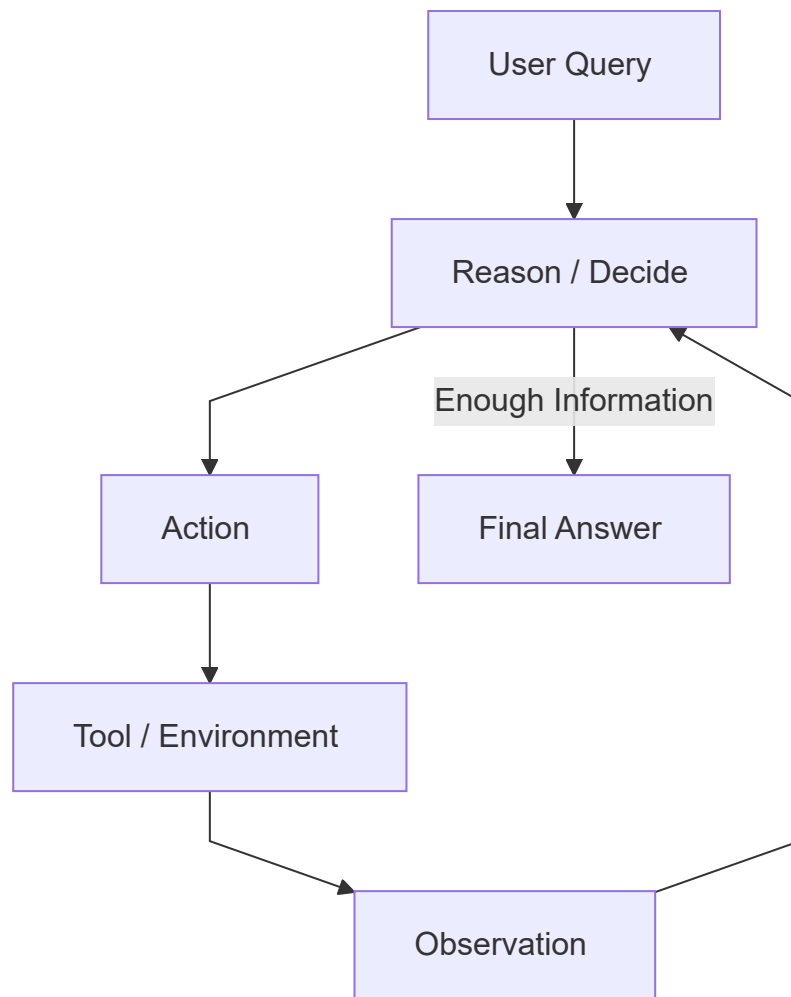
Reason:

Rain is expected, so an umbrella is useful.

Final Answer:

Yes, carrying an umbrella would be advisable...

24. ReAct Architecture



This pattern forms the conceptual basis of many tool-using agents.

25. Important Note About Reasoning

In real applications, we generally care about:

- Decision
- Action
- Observation
- Result

rather than requiring the model to expose private internal reasoning.

A production system may track:

- Tool selected
- Tool arguments
- Tool result
- State transition
- Final response

without exposing hidden chain-of-thought.

26. Autonomy in AI Agents

Agents can have different levels of autonomy.

Low Autonomy

User
↓
Agent Suggests Action
↓
User Approves
↓
Action

Medium Autonomy

User Goal
↓
Agent Executes Safe Tools
↓
Requests Approval for Sensitive Actions

Higher Autonomy

User Goal
↓
Agent Plans
↓
Executes Multiple Actions
↓
Monitors Results
↓
Completes Task

More autonomy is *not automatically better*.

For important actions, human approval can be desirable.

27. Human-in-the-Loop



Human-in-the-loop means the agent pauses and requests human input or approval before continuing.

Example:

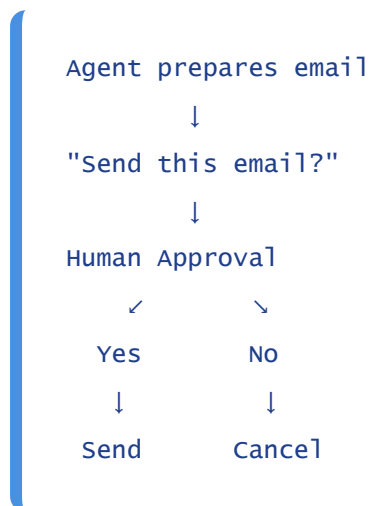
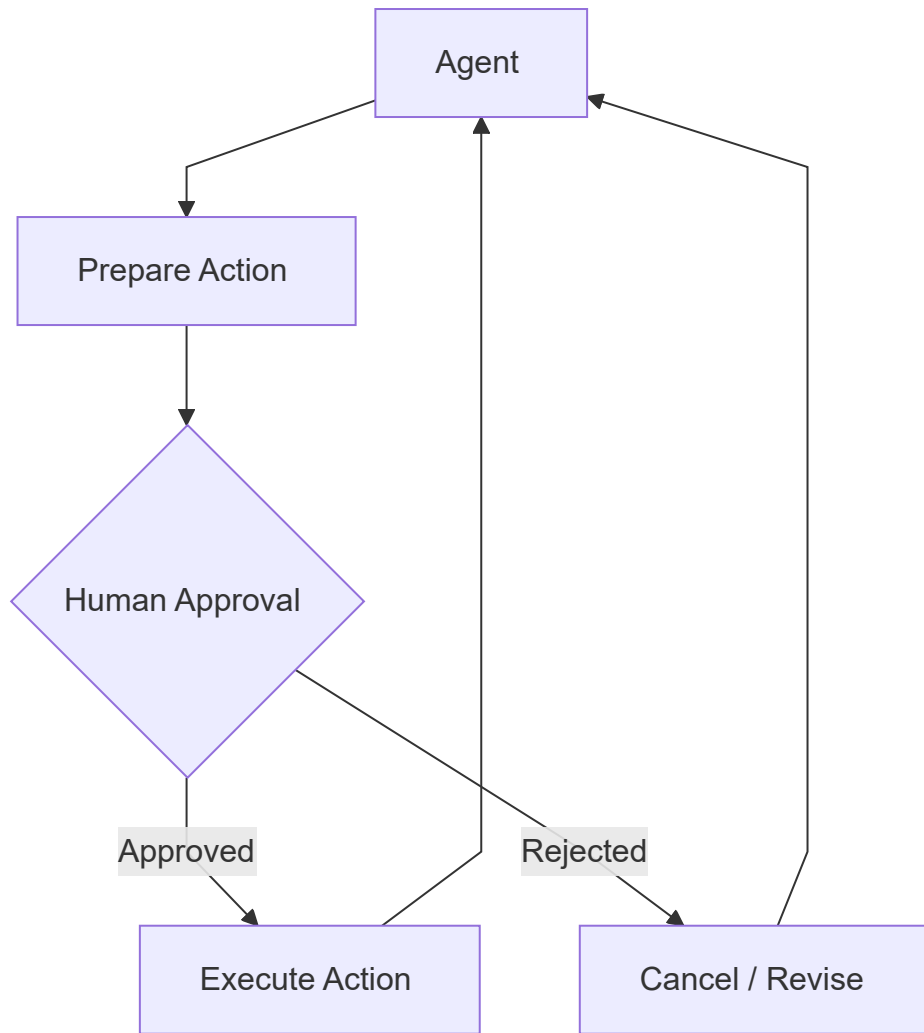


Diagram:



This becomes especially important when actions have real-world consequences.

28. Deterministic Workflow vs Agent



Not every workflow needs an AI agent.

Suppose your process is always:

```
Receive Order
↓
Validate Payment
↓
Update Database
↓
Send Confirmation
```

This is predictable.

Use normal application logic.

An agent becomes useful when the next step depends on interpretation or changing conditions.

Example:

```
User asks complex research question
↓
Agent determines what information is needed
↓
Chooses appropriate tools
↓
Examines results
↓
Chooses next action dynamically
```

29. When Should You Use an Agent?

Use an agent when:

The task requires multiple steps

The next step cannot always be predetermined

Different tools may be needed

Tool results affect future decisions

Natural-language interpretation is important

The workflow requires reasoning over observations

30. When Should You NOT Use an Agent?



Do not use an agent simply because Agentic AI is popular.

Avoid agents when:

A normal function can solve the problem

The workflow is completely deterministic

Simple RAG is sufficient

One API call is sufficient

The task needs strict predictable execution

Agent complexity provides no real benefit

Remember:

Simple Problem
→ Simple Solution

not:

Simple Problem
→ Agent + Memory + 10 Tools + Multi-Agent System

31. RAG vs Agent

You already studied RAG, so this distinction is important.

RAG

Question
↓
Retrieve Documents
↓
Add Context
↓
LLM
↓
Answer

The retrieval pipeline is usually predefined.

Agent

Goal



Agent



Decide What Is Needed



Choose Tool



Observe



Choose Next Step

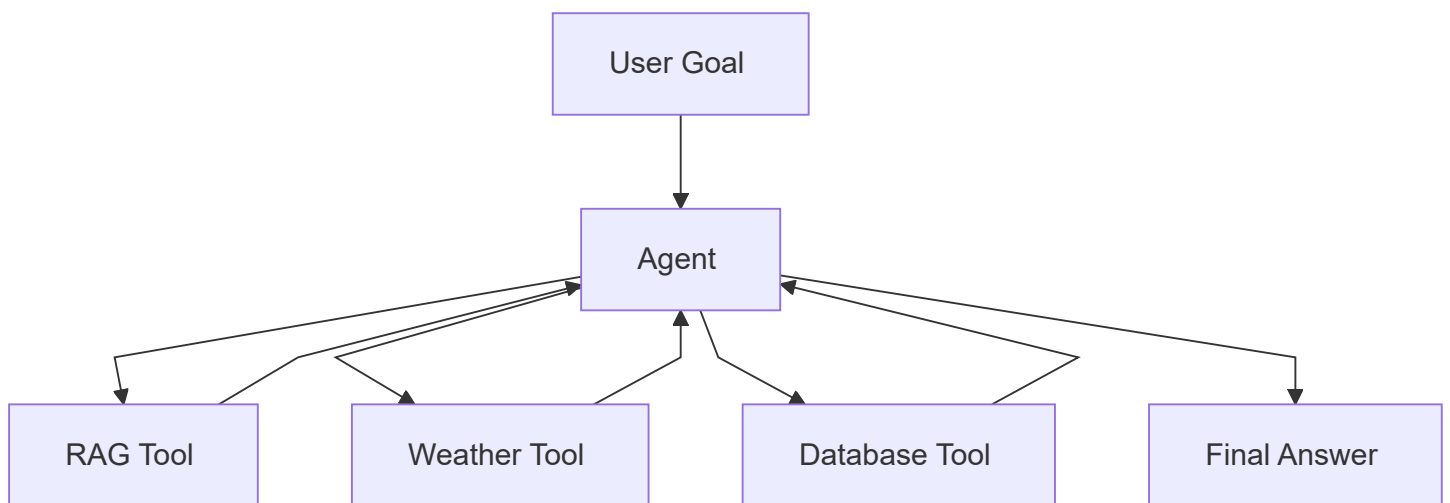


...



Answer

An agent may even use RAG as one of its tools.



So:

RAG

→ A retrieval architecture

Agent

→ A decision-making and action architecture

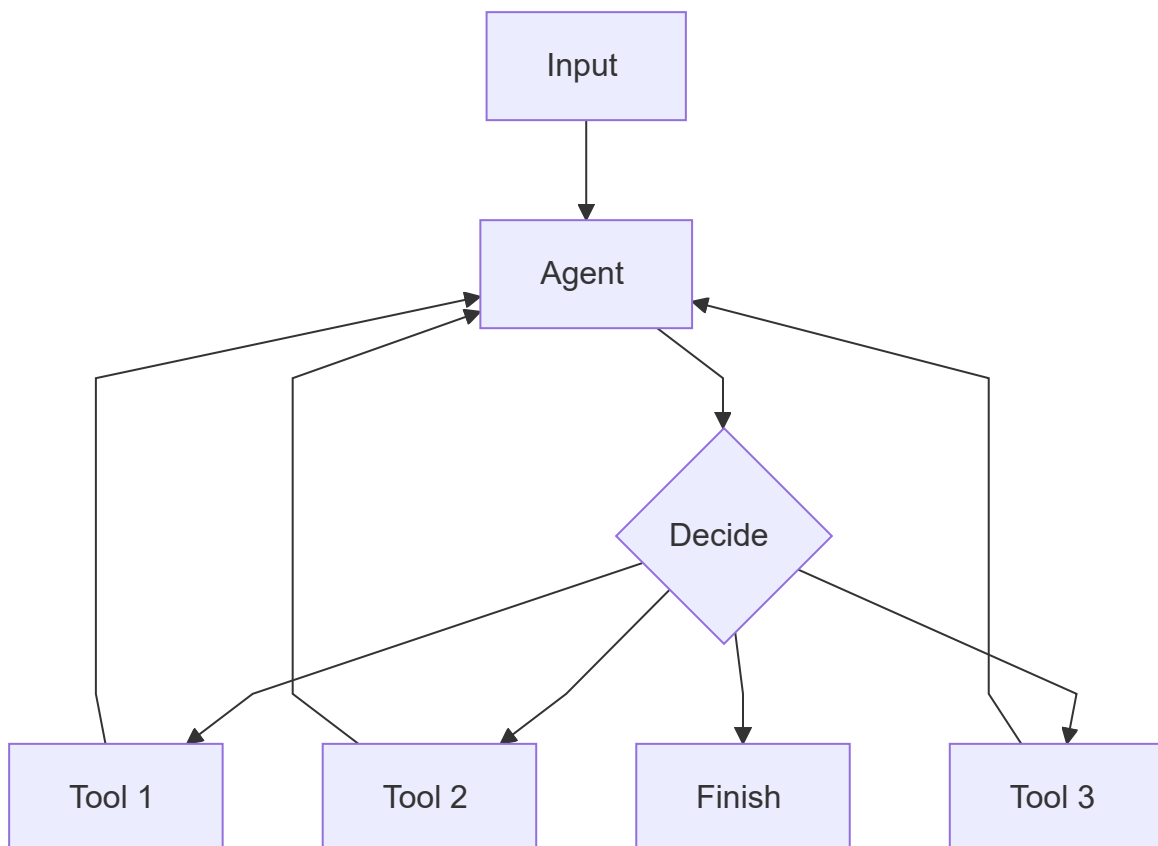
32. Workflow vs Agentic Workflow

Fixed Workflow



Path is predetermined.

Agentic Workflow



The path can change dynamically.

33. First Basic Agent Implementation

Now let's build the **conceptual structure** of a simple agent.

We will intentionally keep it framework-independent first.

Suppose we have two tools:

Weather Tool

Calculator Tool

34. Step 1 – Create Tools

```
def get_weather(city):  
    # In a real application this would call an API.  
  
    weather_data = {  
        "Delhi": "Rainy",  
        "Mumbai": "Cloudy",  
        "Gorakhpur": "Sunny"  
    }  
  
    return weather_data.get(  
        city,  
        "Weather unavailable"  
    )  
  
def calculator(a, b, operation):  
  
    if operation == "add":  
        return a + b
```

```
elif operation == "multiply":  
    return a * b  
  
return "Unsupported operation"
```

These are ordinary Python functions.

The important idea:

```
Tool  
=  
A capability the agent can invoke
```

35. Step 2 – Define Available Tools

Conceptually:

```
tools = {  
    "weather": get_weather,  
    "calculator": calculator  
}
```

Now the agent has access to:

```
weather  
calculator
```

36. Step 3 – Agent Decision

A real LLM-based agent would receive:

```
User Request
+
Agent Instructions
+
Available Tool Descriptions
```

and produce a structured decision such as:

```
{
  "action": "weather",
  "arguments": {
    "city": "Delhi"
  }
}
```

or:

```
{
  "action": "final",
  "answer": "..."
}
```

*This is where **tool calling** becomes important.*

We will implement actual LLM tool calling in Chapter 2.

37. Step 4 – Execute Tool

Our application receives:

```
{  
  "action": "weather",  
  "arguments": {  
    "city": "Delhi"  
  }  
}
```

Then executes:

```
result = get_weather("Delhi")
```

Result:

```
Rainy
```

38. Step 5 – Return Observation

The result is returned to the agent:

```
Tool:  
weather
```

```
Observation:  
Rainy
```

Now the LLM receives this information.

It can decide:

Do I need another action?

OR

Can I answer the user?

39. Step 6 – Agent Loop

Simplified pseudocode:

```
while True:

    decision = agent_decide(
        user_request,
        state,
        tools
    )

    if decision["action"] == "final":

        return decision["answer"]

    tool_name = decision["action"]

    arguments = decision["arguments"]

    result = execute_tool(
        tool_name,
        arguments
    )

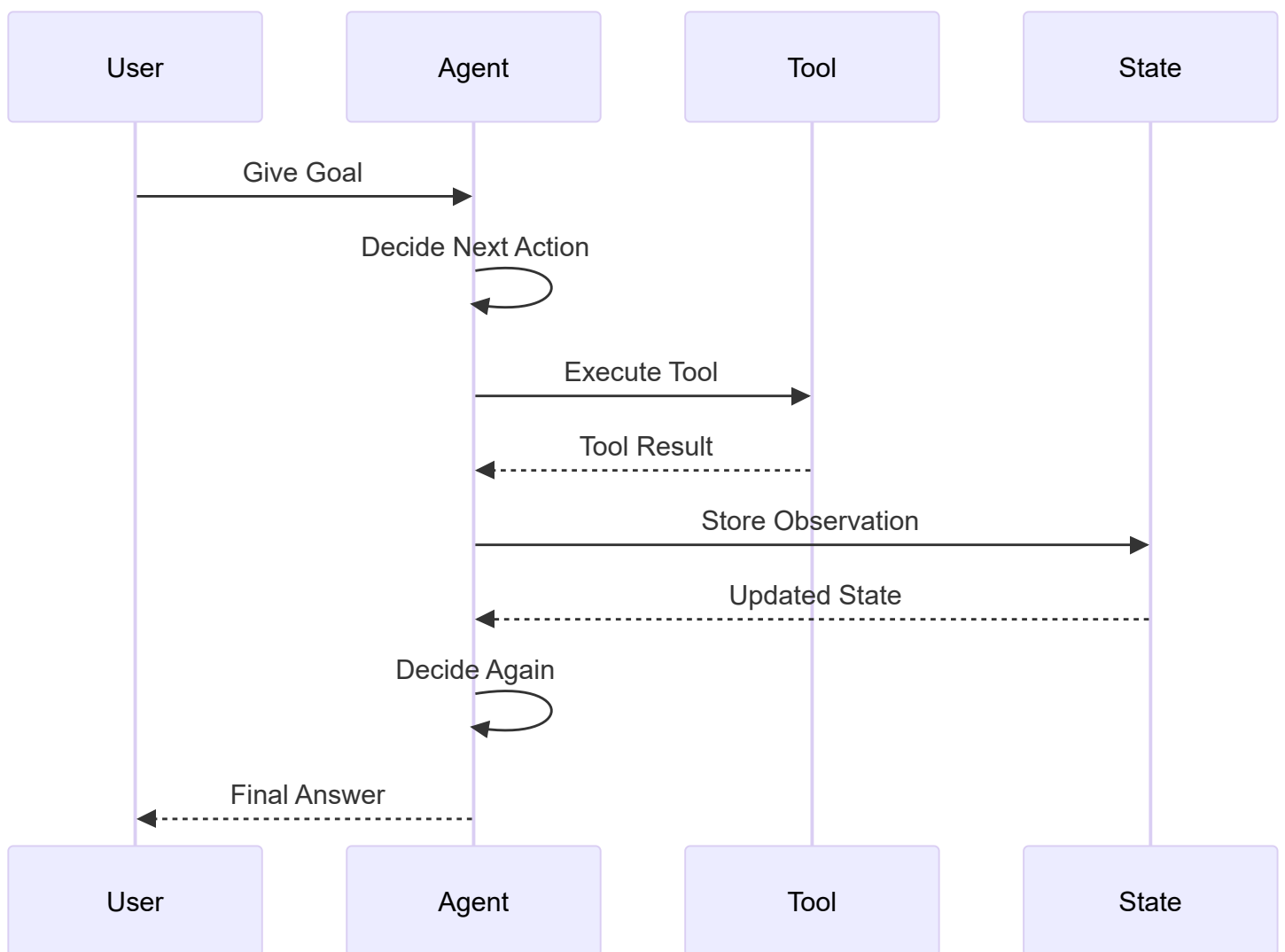
    state.append({
        "action": tool_name,
        "observation": result
```

})

Do not memorize this code.

Understand the architecture.

40. Complete Execution Flow



41. Dry Run

User:

what is the weather in Delhi?

Agent receives

Goal:

Find current Delhi weather.

Available Tools:

weather

calculator

Decision

```
{  
  "action": "weather",  
  "arguments": {  
    "city": "Delhi"  
  }  
}
```

Application executes

```
get_weather("Delhi")
```

Observation

Rainy

Updated state

User:

What is the weather in Delhi?

Action:

weather("Delhi")

Observation:

Rainy

Agent decides again

No more tools required.

Final response

The weather in Delhi is rainy.

That is the basic agent loop.

42. Who Actually Executes the Tool?

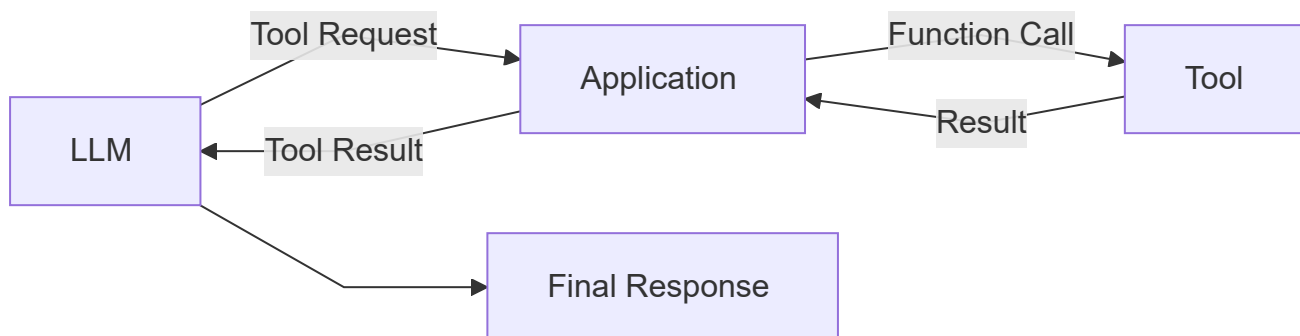
★★★★★ *Very Important Interview Concept*

The LLM usually does *not directly execute your Python function.*

Instead:

```
LLM
↓
Requests Tool Call
↓
Your Application / Agent Runtime
↓
Executes Function
↓
Gets Result
↓
Sends Result Back to LLM
```

Architecture:



This distinction is extremely important.

43. Agent Frameworks

You may encounter frameworks/libraries such as:

LangGraph

LangChain

OpenAI Agents SDK

CrewAI

AutoGen

For our handbook, the main implementation focus will be:

Core Agent Concepts



Tool Calling



State + Memory



LangGraph

Once these concepts are clear, learning other agent frameworks becomes much easier.

44. Why LangGraph Later?

Our basic agent currently looks like:

```
graph TD; A[Agent] --> B[Tool]; B --> C[Agent]; C --> D[Tool]; D --> E[Agent]; E --> F[Final Answer]
```

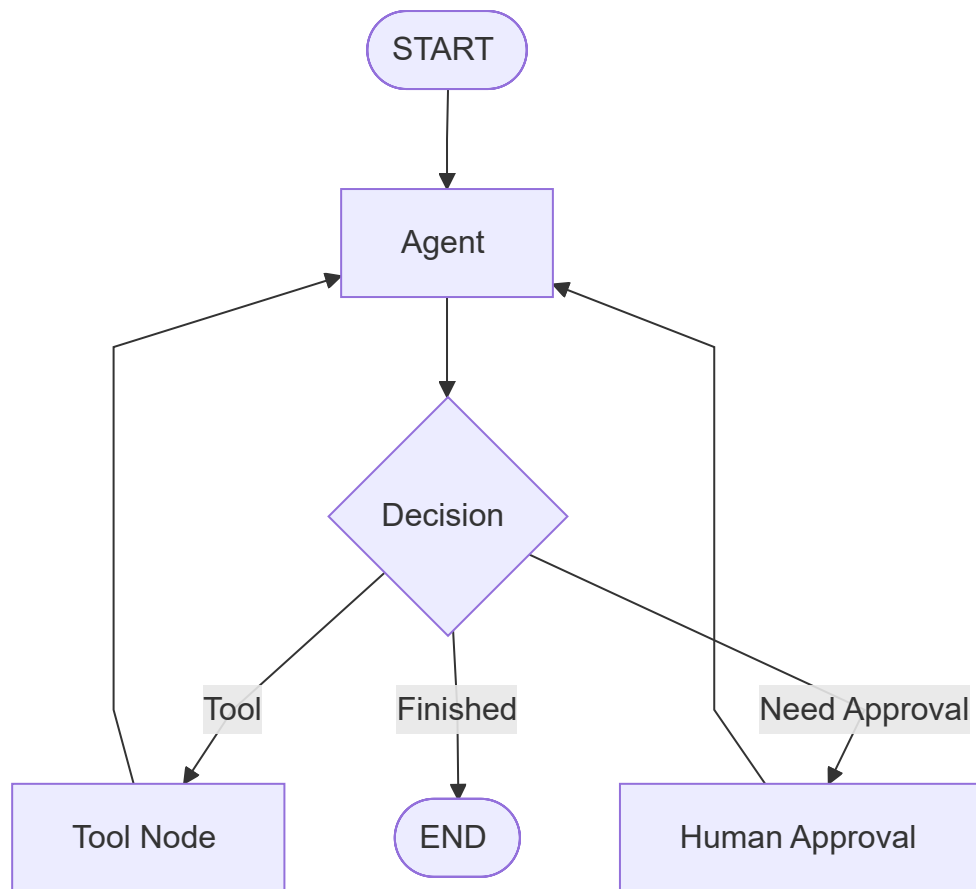
Agent
↓
Tool
↓
Agent
↓
Tool
↓
Agent
↓
Final Answer

As workflows become larger, we need explicit control over:

- State
- Branches
- Loops
- Retries
- Persistence
- Human Approval
- Tool Routing

This is where graph-based orchestration becomes useful.

Example:



We will build this properly in Chapter 4.

45. Common Beginner Misconceptions

Misconception 1

Agent = LLM

Incorrect.

Better mental model:

Agent
=
LLM
+
Instructions
+
Tools
+
State
+
Control Logic

Misconception 2

Every chatbot is an agent.

Incorrect.

A basic:

Prompt → LLM → Response

chatbot is not necessarily agentic.

Misconception 3

Agent must use many tools.

No.

An agent can have one tool or many tools.

Misconception 4

More autonomy = better system.

No.

Higher autonomy also introduces:

More uncertainty

More failure possibilities

More security concerns

More cost

Harder debugging

Misconception 5

Agentic AI replaces RAG.

No.

An agent can use RAG as a capability.

Misconception 6

Agents always need multiple agents.

No.

A single well-designed agent is often enough.

46. Important Interview Questions

Q1. What is Agentic AI? ★ ★ ★ ★ ★

Answer:

Agentic AI refers to AI systems that can work toward goals by making decisions, taking actions through tools, observing the results and dynamically deciding what to do next.

Q2. What is an AI agent? ★ ★ ★ ★ ★

Answer:

An AI agent is a goal-directed system that typically combines an LLM with instructions, tools, state or memory, and control logic so that it can make decisions and perform actions rather than only generate text.

Q3. LLM vs AI Agent? ★ ★ ★ ★ ★

Answer:

An LLM primarily generates outputs from provided context, while an AI agent uses an LLM as a reasoning or decision component and can additionally interact with tools, maintain state and execute multi-step workflows.

Q4. What are the main components of an AI agent? ★ ★ ★ ★ ★

Answer:

The main components are generally:

LLM / Model

Goal / Instructions

Tools

State

Memory

Control Logic

Q5. What is an agent loop? ★ ★ ★ ★ ★

Answer:

An agent loop is the repeated process in which an agent decides what to do, performs an action, observes the result and uses that observation to determine the next action until the task is complete.

Q6. What is ReAct? ★ ★ ★ ★ ★

Answer:

ReAct stands for Reasoning and Acting. It is an agent pattern where reasoning or decision-making is interleaved with actions and observations, allowing the system to adapt its next step based on external results.

Q7. What is an action?

Answer:

An action is an operation selected by an agent, such as calling an API, searching a database or invoking another tool.

Q8. What is an observation?

Answer:

An observation is the result returned after an agent performs an action, which can then influence the agent's next decision.

Q9. What is state in an AI agent? ★ ★ ★ ★

Answer:

State is the information maintained during execution that represents the current condition of the workflow, such as messages, intermediate results, tool outputs and current progress.

Q10. State vs memory?

Answer:

State mainly represents information required for the current workflow execution, while memory refers to information retained and retrieved across interactions or over longer periods.

Q11. Why do agents need tools? ★ ★ ★ ★ ★

Answer:

Tools allow agents to interact with external systems and perform capabilities that an LLM cannot inherently perform, such as retrieving live information, querying databases, performing calculations or triggering application actions.

Q12. Does the LLM execute tools directly?



Answer:

Usually no. The LLM generates a structured tool-call request, while the application or agent runtime executes the actual function and returns the result to the LLM.

Q13. RAG vs AI Agent? ★ ★ ★ ★ ★

Answer:

RAG is primarily an architecture for retrieving relevant external knowledge and supplying it to an LLM. An AI agent is a decision-making system that can dynamically choose and execute actions. RAG itself can be exposed as one of an agent's tools.

Q14. When should you use an AI agent?

Answer:

Agents are useful when a task requires dynamic multi-step decisions, interaction with multiple tools or systems, and when later actions depend on observations from earlier actions.

Q15. When should you avoid agents?



Answer:

Agents should be avoided when a simple deterministic workflow, direct API call, standard application logic or basic RAG pipeline can solve the problem reliably with less complexity.

Q16. What is human-in-the-loop?

Answer:

Human-in-the-loop is an architecture where the agent pauses at certain points and requires human review, input or approval before continuing with an action.

47. Final Mental Model



Remember the progression:

NORMAL LLM

User



Prompt



LLM



Response

Then:

TOOL-USING LLM

User



LLM



Tool



Tool Result



LLM



Response

Then:

AI AGENT

Goal



Agent



Decide



Action



Observation



Update State



Decide Again



...



Final Answer

And finally:

AGENTIC AI SYSTEM

User Goal



Agent



Decision Logic



Tools



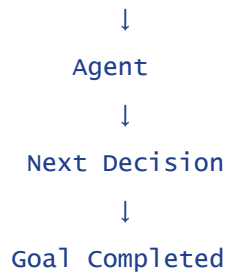
Memory



State



Observation



The most important formula to remember is:

AI Agent
=
LLM
+
Goal / Instructions
+
Tools
+
State / Memory
+
Control Loop

And the most important loop is:

Decide
↓
Act
↓
Observe
↓
Decide Again

An AI agent extends an LLM from simply generating responses into a goal-directed system capable of choosing actions, interacting with external tools, observing results and dynamically continuing until the task is complete.

Agentic AI Mastery Handbook

Chapter 1 – Agentic AI Fundamentals

Part 2 – Agent Design, Planning, Execution & First Complete Agent

Interview Importance: ★ ★ ★ ★ ★

Difficulty: Beginner → Intermediate

Prerequisites: Chapter 1 – Part 1

1. From LLM to Complete AI Agent

In Part 1, we learned:

AI Agent

=

LLM

+

Instructions

+

Tools

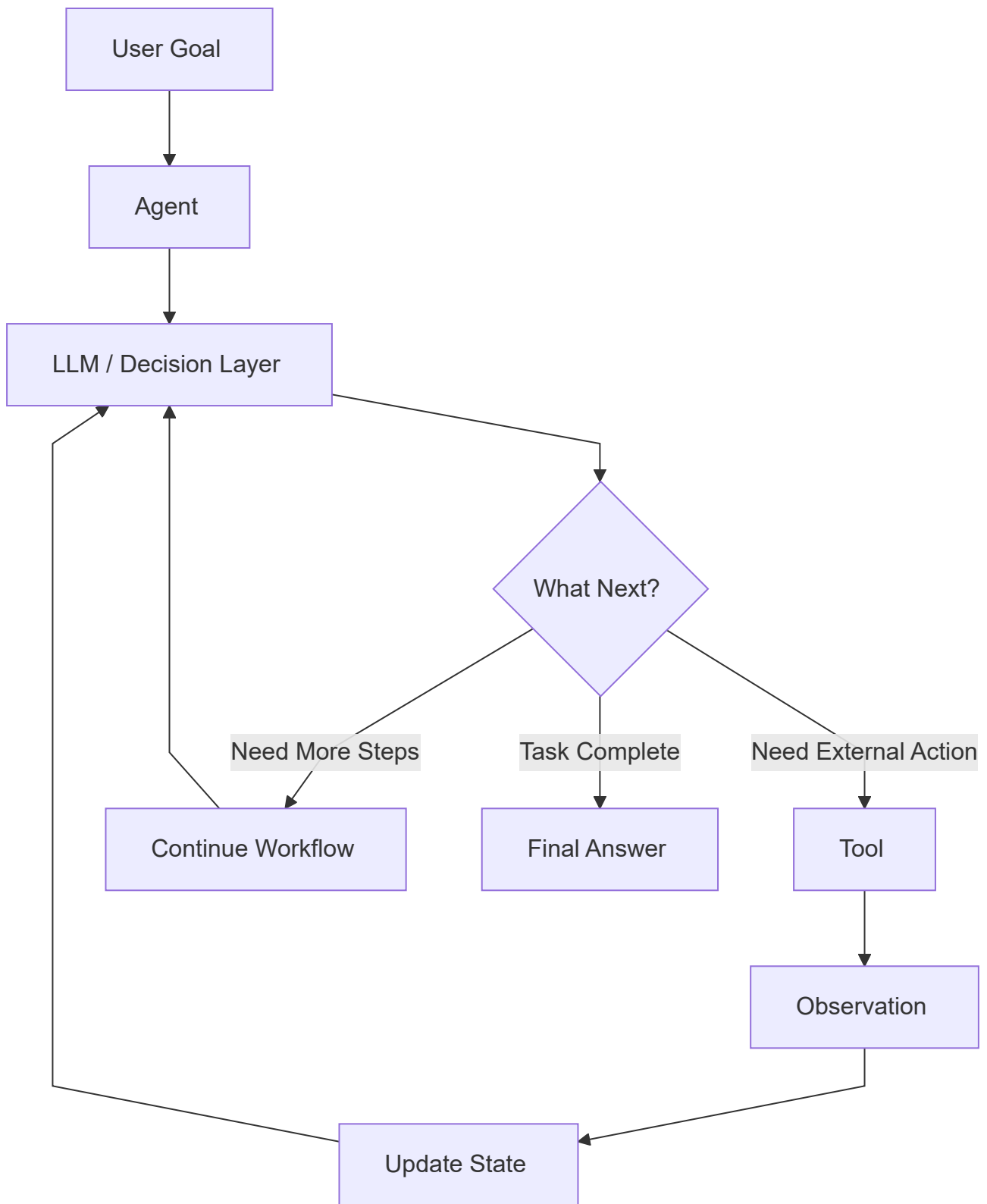
+

State / Memory

+

Control Loop

Now let's understand how these components work together in a complete system.



The agent continuously answers one important question:

What should I do next to achieve the user's goal?

2. Goal-Oriented Behavior



A normal LLM primarily responds to a prompt.

An agent works toward a:

Goal

Example:

User:

Find the current weather in Delhi
and recommend whether I should
plan an outdoor event.

The goal is not simply:

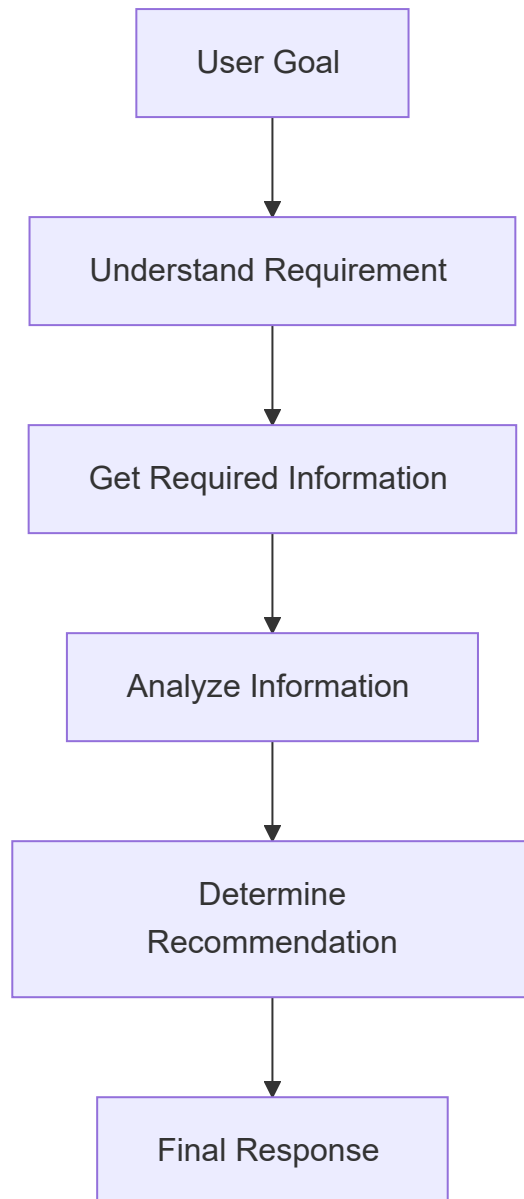
Generate text

The goal is:

Obtain Current Weather
+
Analyze Conditions
+
Provide Recommendation

3. Goal → Tasks → Actions

A complex goal may require multiple tasks.



Some tasks may require tools.

For example:

Get Current Weather

→ Weather API

Perform Calculation

→ Calculator

Find User Information

→ Database

Search Documents

→ RAG / Search Tool

4. Task Decomposition



Task decomposition means breaking a larger goal into smaller manageable tasks.

Example:

Goal:

Compare the weather of Delhi and Mumbai
and tell me which city is better for an
outdoor event today.

Possible decomposition:

Task 1

→ Get Delhi weather

Task 2

→ Get Mumbai weather

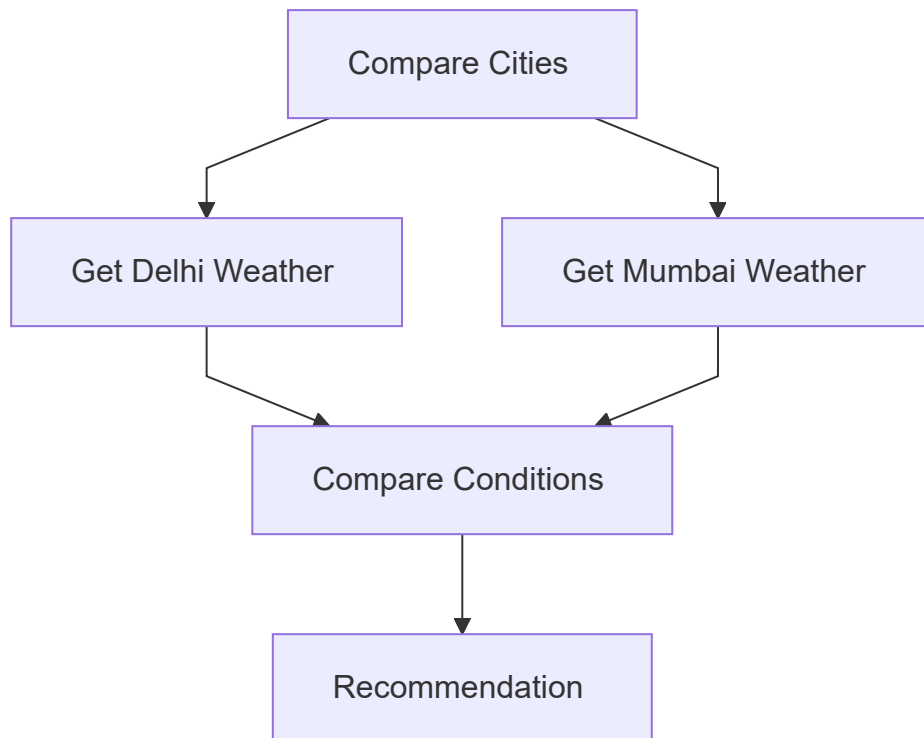
Task 3

→ Compare conditions

Task 4

→ Recommend city

Diagram:



5. Planning in AI Agents



Planning is the process of determining the steps or actions required to achieve a goal.

Conceptually:

```
graph TD; A[Goal] --> B[Understand Requirements]; B --> C[Determine Required Steps]; C --> D[Execute Steps]; D --> E[Observe Results]; E --> F[Adjust Plan if Required]; F --> G[Complete Goal];
```

Goal
↓
Understand Requirements
↓
Determine Required Steps
↓
Execute Steps
↓
Observe Results
↓
Adjust Plan if Required
↓
Complete Goal

6. Fixed Plan vs Dynamic Planning

There are two useful ways to think about planning.

Fixed Workflow

Steps are known beforehand.

```
Step 1
↓
Step 2
↓
Step 3
↓
Final Result
```

Example:

```
Get weather
↓
Format weather
↓
Display weather
```

*This often does **not need an agent**.*

Dynamic Planning

The next step depends on previous results.

```
Goal
↓
Action 1
↓
Observation
↓
Choose Action 2
↓
Observation
↓
Choose Next Action
```

Example:

```
graph TD; A[Search product] --> B[Product unavailable]; B --> C[Search alternative]; C --> D[Alternative too expensive]; D --> E[Search another option];
```

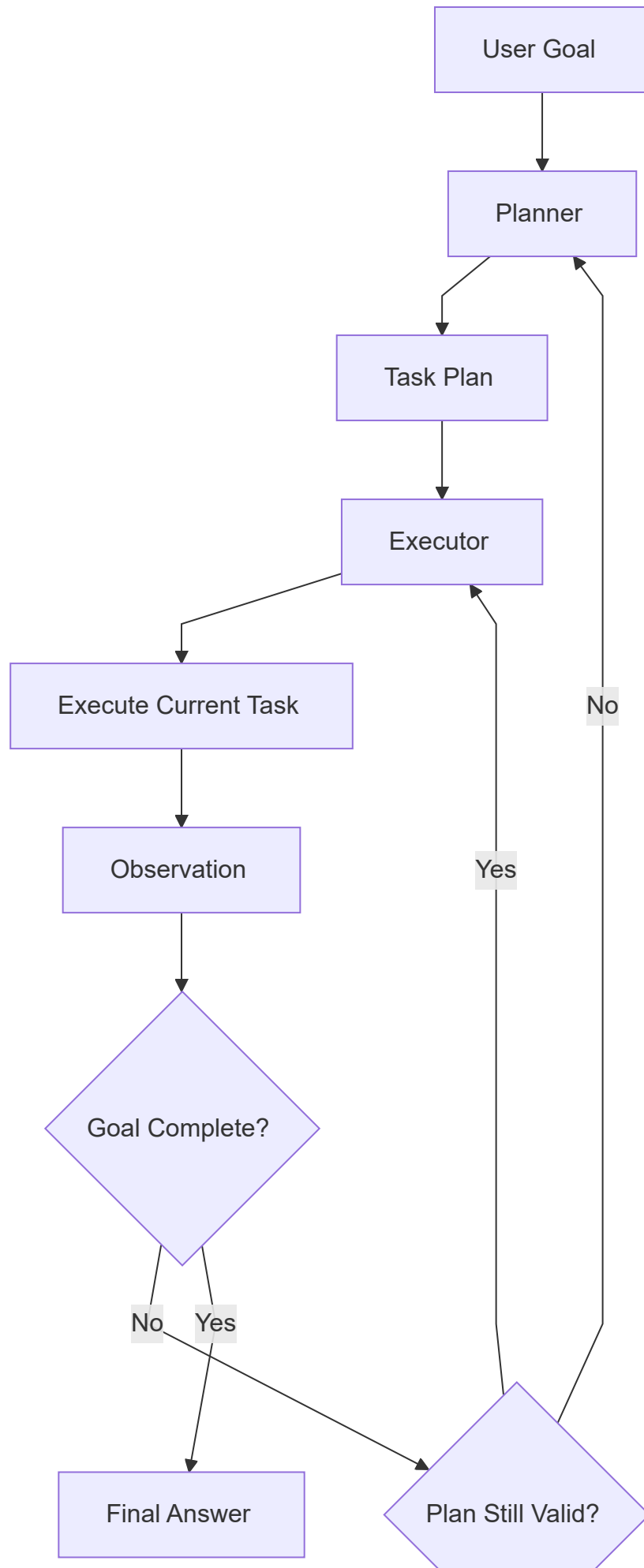
This is more agentic.

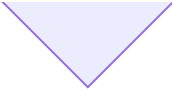
7. Plan-and-Execute Pattern

A common conceptual agent pattern is:

```
graph TD; A[Planner] --> B[Create Plan]; B --> C[Executor]; C --> D[Execute Tasks]; D --> E[Observe Results]; E --> F[Update / Replan]; F --> G[Final Result];
```

Architecture:





8. Planner vs Executor

Planner

Responsible for:

- Understanding the goal
 - Breaking goal into tasks
 - Determining task order
 - Changing plan when necessary
-

Executor

Responsible for:

- Executing selected task
- Calling tools
- Collecting results
- Returning observations

Conceptually:

Planner

→ What should be done?

Executor

→ Perform it

These do not necessarily have to be separate LLMs.

A single agent can perform both roles.

9. Agent Decision Cycle

At every step, the agent may decide among:

Use a Tool

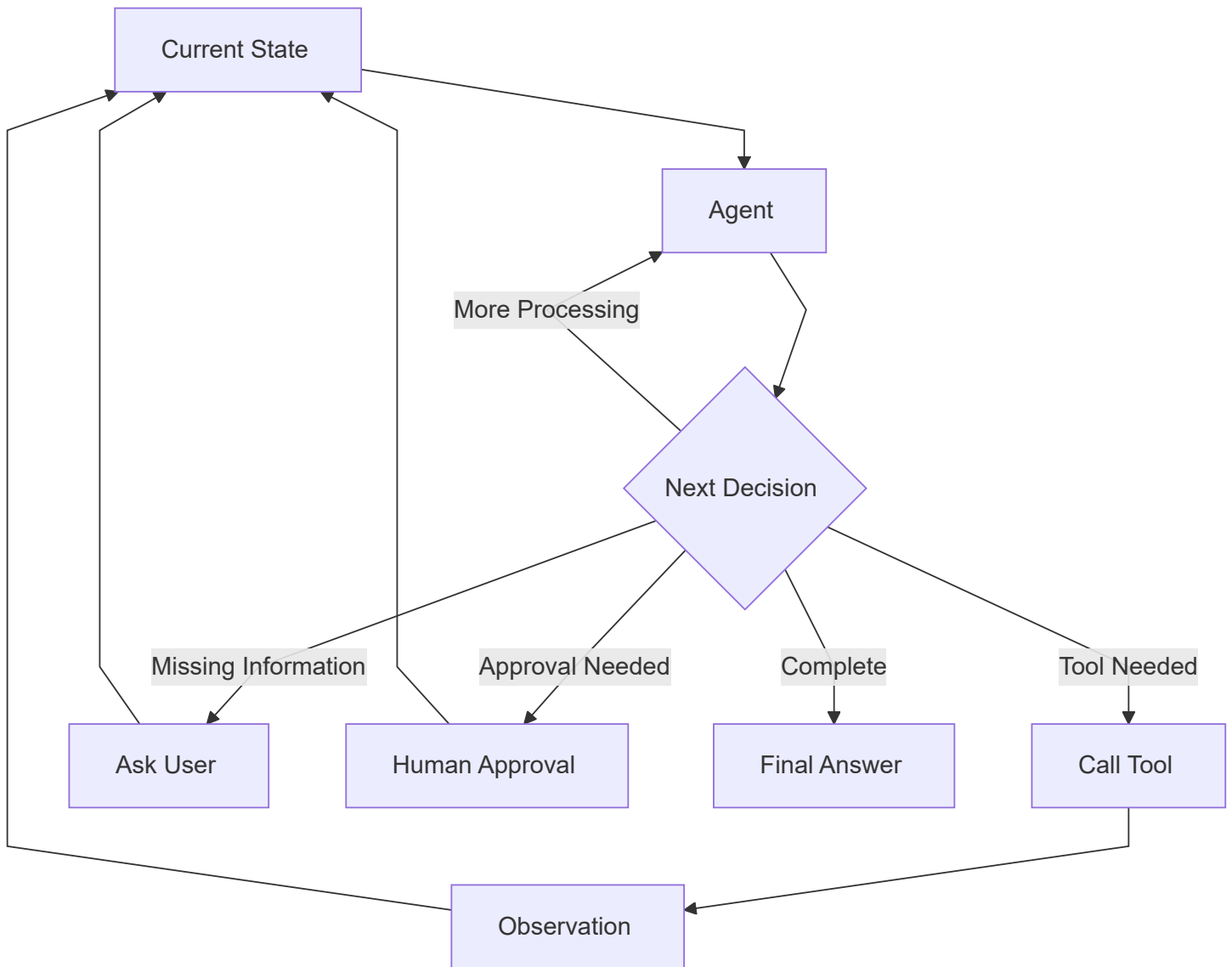
Continue Reasoning / Processing

Ask User for Missing Information

Request Human Approval

Finish Task

Diagram:



10. Environment

An agent operates within an:

Environment

The environment contains systems the agent can interact with.

Examples:

Internet

Application Backend

Database

File System

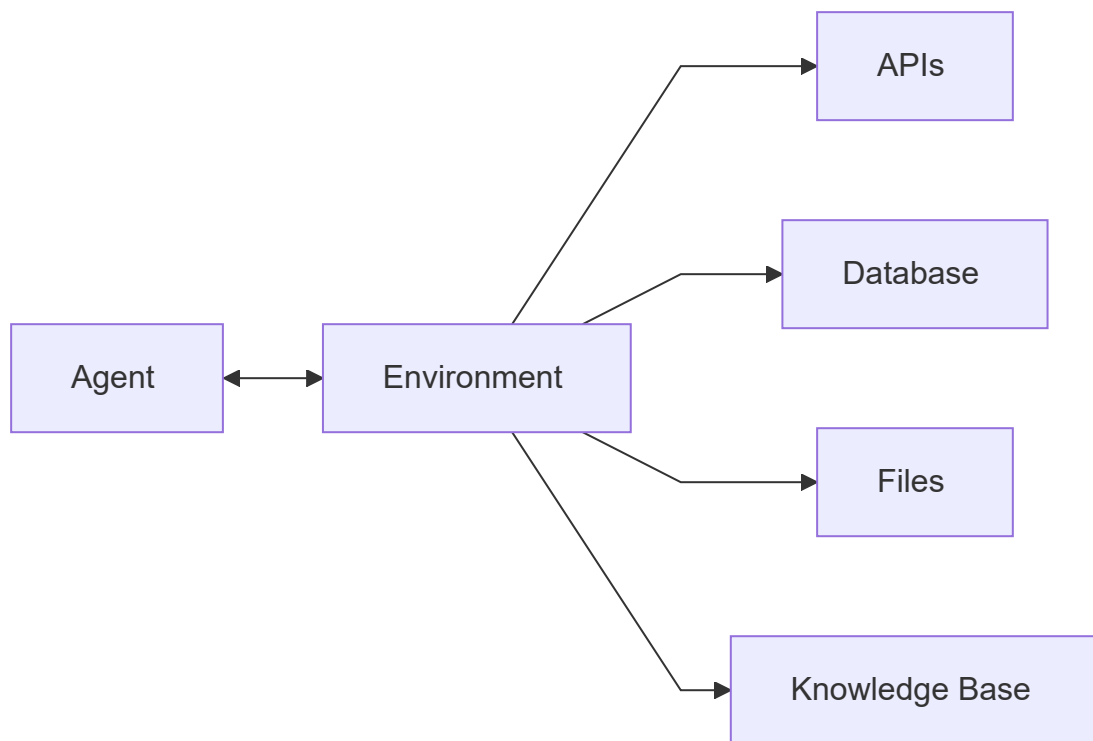
External APIs

Email System

Calendar

Knowledge Base

Architecture:



The agent interacts with the environment primarily through controlled tools.

II. Agent Perception

An agent needs information about what happened after an action.

This can be thought of as:

Perception / Observation

Example:

Action:

```
search_product("Laptop")
```

Observation:

3 products found

The observation becomes new information for the agent.

Action

↓

Environment

↓

Observation

↓

Update State

↓

Next Decision

12. Agent State During Execution

Suppose the user asks:

```
Compare Delhi and Mumbai weather.
```

Initial state:

```
state = {  
  "goal": "Compare Delhi and Mumbai weather",  
  "delhi_weather": None,  
  "mumbai_weather": None  
}
```

After first tool:

```
state = {  
  "goal": "Compare Delhi and Mumbai weather",  
  "delhi_weather": "Rainy",  
  "mumbai_weather": None  
}
```

After second tool:

```
state = {  
  "goal": "Compare Delhi and Mumbai weather",  
  "delhi_weather": "Rainy",  
  "mumbai_weather": "Cloudy"  
}
```

Now the agent has enough information to compare them.

13. State Changes Over Time



This concept becomes extremely important in LangGraph.

14. Agent Control Loop



A simplified agent control loop can be represented as:

```
while not goal_completed:

    decision = decide_next_action(state)

    result = execute(decision)

    state = update_state(
        state,
        result
    )
```

Finally:

```
return generate_final_answer(state)
```

This is only conceptual pseudocode.

The important flow is:

```
State
↓
Decision
↓
Action
↓
Observation
↓
State Update
↓
Repeat
```

15. Stopping Condition



An agent must know when to stop.

Otherwise:

```
Agent
↓
Tool
↓
Agent
↓
Tool
↓
Agent
↓
Tool
↓
...
```

could continue unnecessarily.

A:

Stopping Condition

determines when execution should finish.

Examples:

Goal completed

Required information obtained

Final answer generated

Maximum number of steps reached

Timeout reached

Human cancels execution

16. Maximum Steps

A simple safety mechanism is:

```
MAX_STEPS = 10
```

Then:

```
for step in range(MAX_STEPS):  
  
    decision = agent(state)  
  
    if decision == "finish":  
        break  
  
    execute_action()
```

This helps prevent:

- Infinite loops
- Excessive API calls
- Unexpected cost
- Very long execution

17. Agent Failure Example

Suppose:

```
Agent  
↓  
Search Tool  
↓  
No Result  
↓  
Agent  
↓  
Search Tool  
↓  
No Result  
↓
```

```
Agent
↓
Search Tool
↓
...
```

Without proper stopping logic, the agent can repeatedly perform the same unsuccessful action.

Better:

```
Search Attempt 1
↓
Failed

Search Attempt 2
↓
Failed

Retry Limit Reached
↓
Use Alternative / Ask User / Stop
```

18. Retry Mechanism



Tools can fail because of:

Network errors

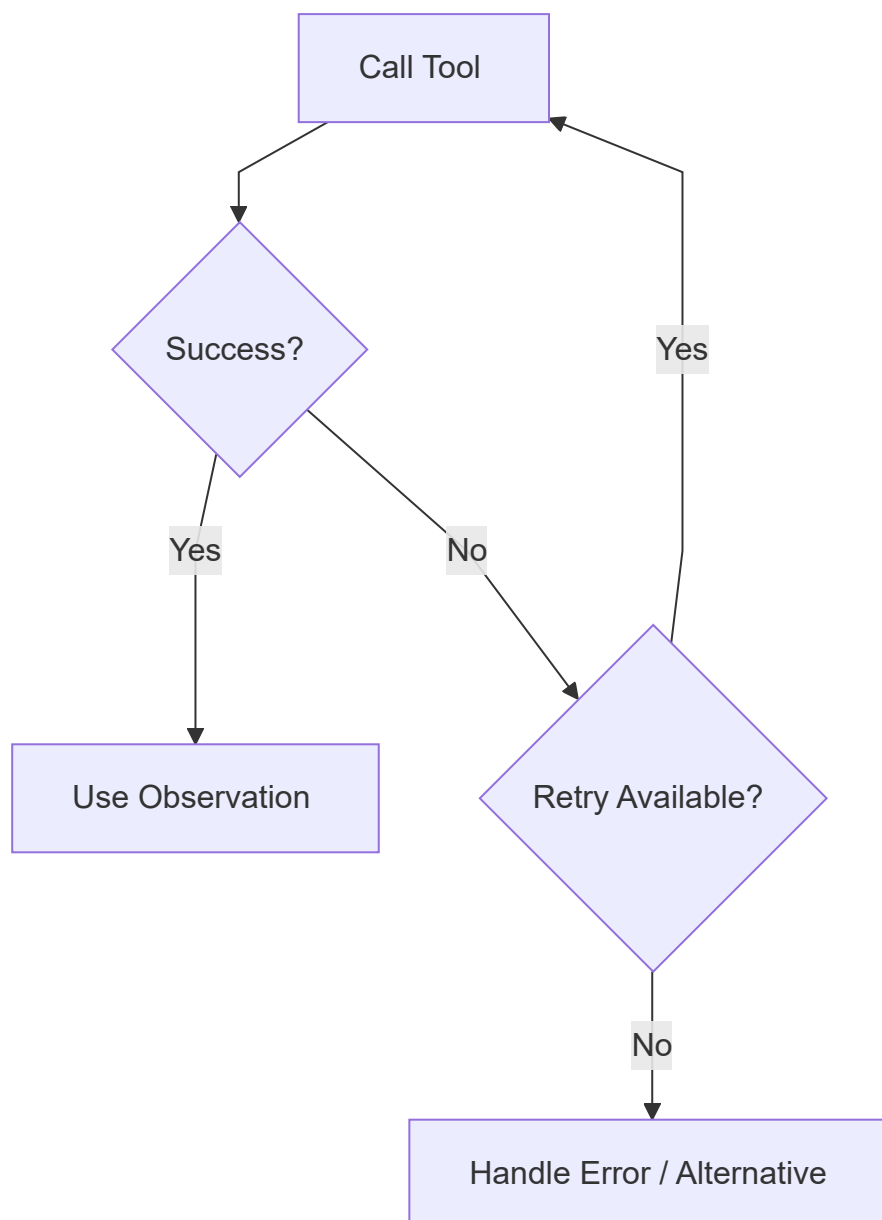
API timeout

Rate limits

Invalid arguments

Temporary service failure

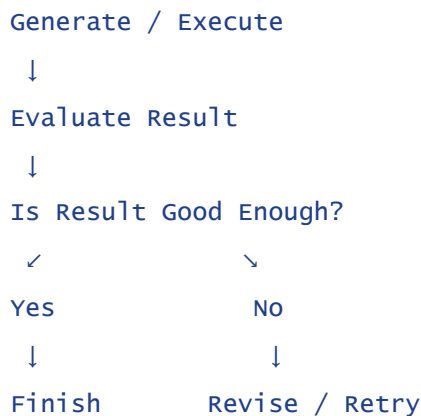
A controlled retry strategy may help.



19. Reflection / Self-Correction

Agents may inspect the result of an action and determine whether another attempt is required.

Conceptually:



Example:

Goal:
Find information about company X.

Search Result:
Irrelevant information.

Agent:
This result doesn't answer the question.

Action:
Search again with a better query.

This is a form of:

Reflection / Self-Correction

20. Reflection Does Not Guarantee Correctness

Important:

Reflection
≠
Guaranteed Correct Answer

An LLM can incorrectly judge its own output.

Therefore production systems may additionally use:

Validation

Deterministic checks

External evaluators

Human approval

Retry limits

21. Deterministic vs LLM Decisions

A strong agentic system does **not** ask the LLM to decide everything.

Example:

```
Is payment amount > ₹10,000?
```

This can be handled with normal code:

```
if amount > 10000:  
    require_approval()
```

Do not ask an LLM to make a deterministic numerical decision unnecessarily.

22. Hybrid Agent Architecture



Good agent systems combine:

```
LLM Decisions  
+  
Deterministic Code
```

Use LLM for:

Natural-language understanding

Choosing among tools

Interpreting unstructured information

Flexible planning

Generating responses

Use normal code for:

Authentication

Permissions

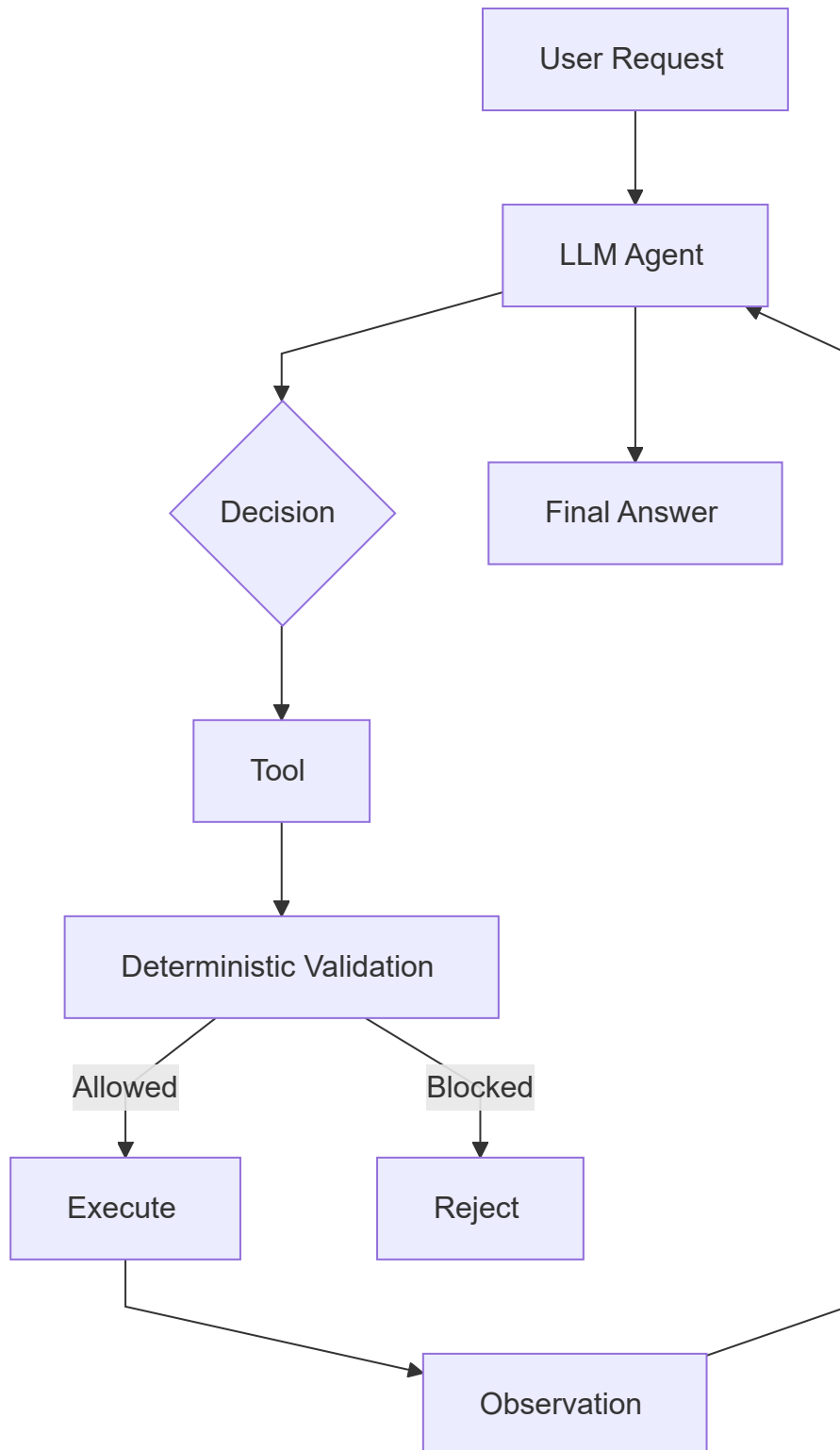
Numerical validation

Limits

Business rules

Critical safety checks

Architecture:



23. Agent Instructions



Agents require clear instructions.

Example:

You are a weather assistant.

Your responsibilities:

1. Answer weather-related questions.
2. Use the weather tool whenever current weather information is required.
3. Never invent weather information.
4. If the location is missing, ask the user.
5. Do not use tools unrelated to the request.
6. Give a concise final response.

Good instructions improve:

Tool selection

Consistency

Reliability

Safety

Response quality

24. Tool Descriptions Matter

Suppose an agent has:

```
Tool A:  
search()
```

```
Tool B:  
lookup()
```

These descriptions are unclear.

Better:

```
search_web(query)
```

```
Use this tool to search the internet for  
current or external information.
```

and:

```
get_weather(city)
```

```
Use this tool to retrieve current weather  
information for a city.
```

The LLM uses tool descriptions when deciding which tool to call.

25. Agent Input

An agent may receive more than the user's message.

Conceptually:

System Instructions

+

User Goal

+

Available Tools

+

Current State

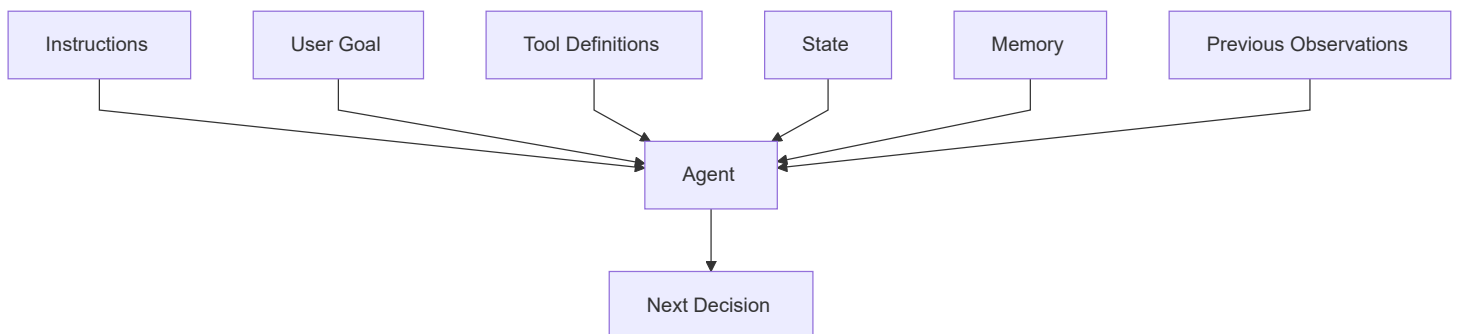
+

Previous Tool Results

+

Relevant Memory

All of these help the model decide the next step.



26. Agent Output

The agent's output is not always the final natural-language answer.

It might instead be:

```
Tool Call  
  
Ask User  
  
Request Approval  
  
Continue Workflow  
  
Final Response
```

Example structured tool request:

```
{  
  "tool": "get_weather",  
  "arguments": {  
    "city": "Delhi"  
  }  
}
```

This will become very important in Chapter 2.

27. First Complete Agent Architecture

Let's design a small:

Weather Recommendation Agent

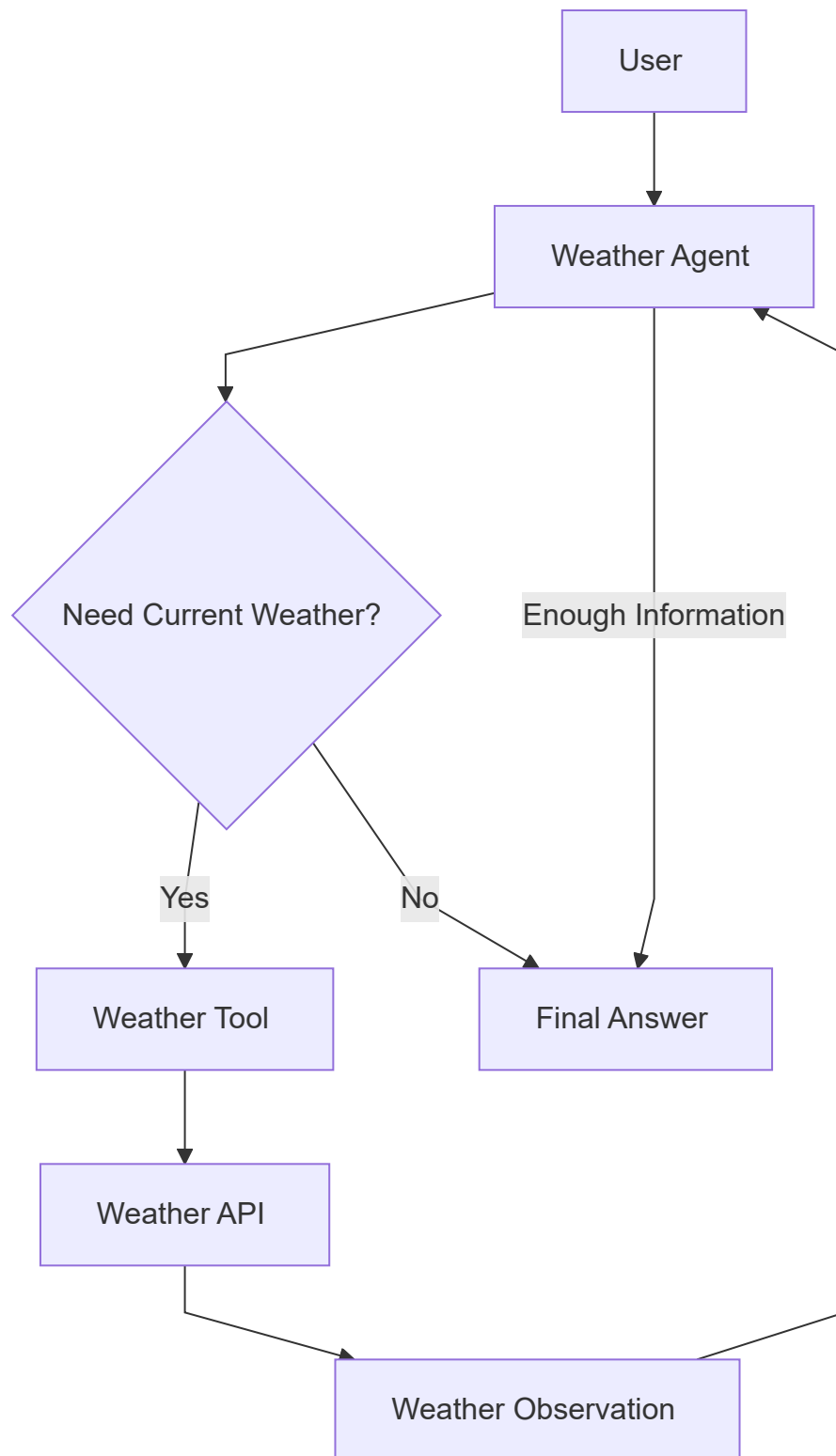
Goal:

Answer current weather questions
and make simple weather-based recommendations.

Available tool:

`get_weather(city)`

Architecture:



28. Step 1 – Create Weather Tool

For learning, we'll use a simple mock tool.

```
def get_weather(city):  
  
    weather = {  
        "delhi": {  
            "temperature": 31,  
            "condition": "rainy"  
        },  
  
        "mumbai": {  
            "temperature": 29,  
            "condition": "cloudy"  
        }  
    }  
  
    return weather.get(  
        city.lower(),  
        {  
            "error": "Weather unavailable"  
        }  
    )
```

In a real application:

```
get_weather()  
↓  
Weather API
```

29. Step 2 – Define Agent State

```
state = {  
    "user_request": None,  
    "tool_result": None,  
    "steps": 0,  
    "completed": False  
}
```

This stores the current workflow information.

30. Step 3 – Agent Decision Function

For now, we'll simulate the decision logic.

Actual LLM-based tool calling comes in Chapter 2.

```
def decide_action(state):  
  
    request = state["user_request"].lower()  
  
    if state["tool_result"] is None:  
  
        if "weather" in request:  
  
            return {  
                "action": "get_weather",  
                "city": "delhi"  
            }  
  
    return {  
        "action": "finish"
```

```
}
```

Again:

This is only to understand the agent architecture.

Later the **LLM itself will choose the tool and arguments.**

31. Step 4 – Execute Action

```
def execute_action(decision):  
  
    if decision["action"] == "get_weather":  
  
        return get_weather(  
            decision["city"]  
        )  
  
    return None
```

32. Step 5 – Agent Loop

```
MAX_STEPS = 5  
  
def run_agent(user_request):  
  
    state = {  
        "user_request": user_request,  
        "tool_result": None,  
        "steps": 0,
```

```
        "completed": False
    }

    while state["steps"] < MAX_STEPS:

        state["steps"] += 1

        decision = decide_action(state)

        if decision["action"] == "finish":

            state["completed"] = True

            break

        result = execute_action(
            decision
        )

        state["tool_result"] = result

    return state
```

33. Running the Agent

```
result = run_agent(
    "what is the weather in Delhi?"
)

print(result)
```

Conceptual result:

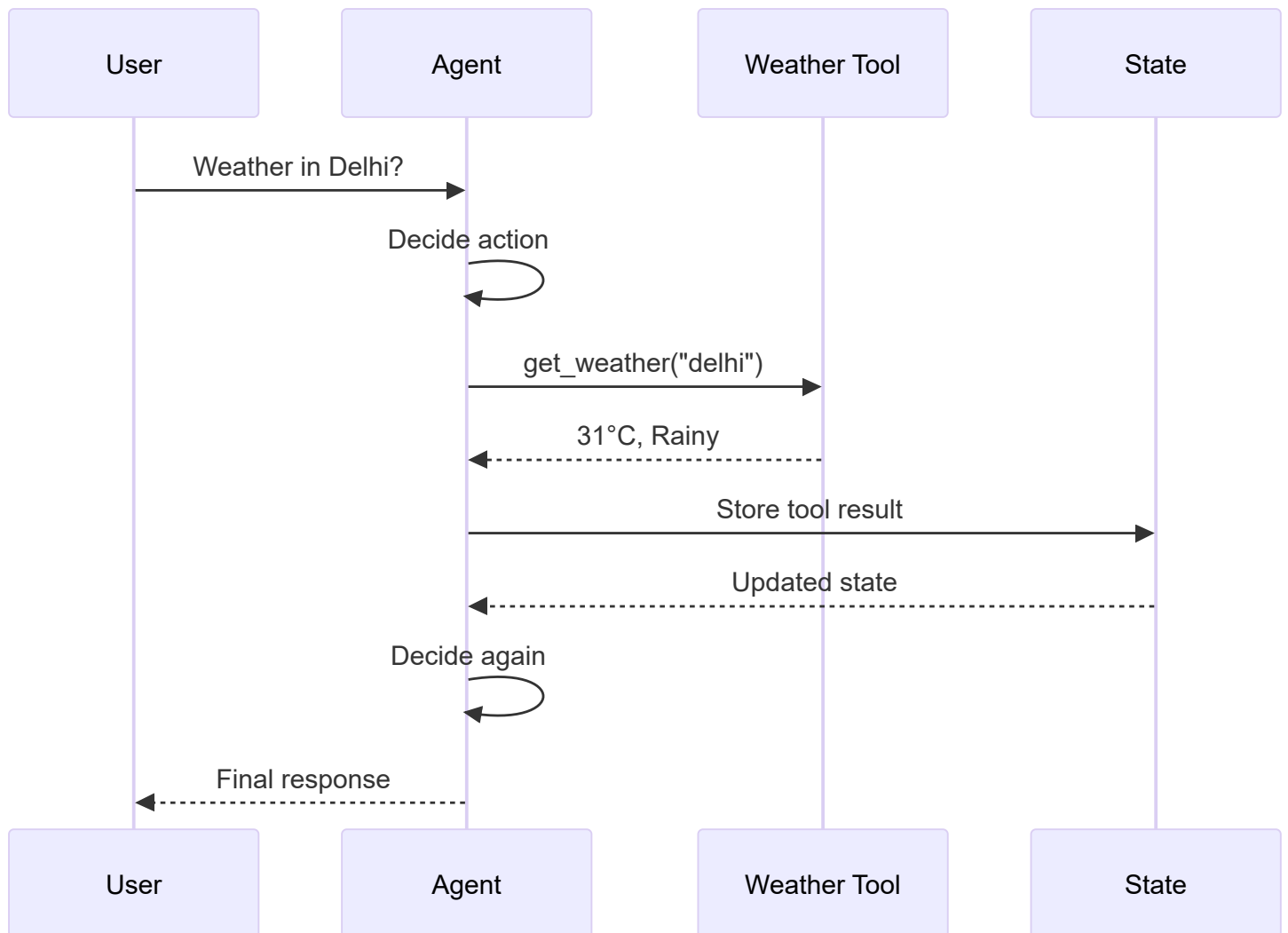
```
{
  "user_request":
    "what is the weather in Delhi?",

  "tool_result": {
    "temperature": 31,
    "condition": "rainy"
  },

  "steps": 2,

  "completed": True
}
```

34. Execution Flow



35. What Is Missing From This Agent?

Our implementation is currently:

Rule-Based Decision

+

Tool

+

State

+

Loop

The major missing piece is:

Actual LLM Tool Calling

Currently:

```
if "weather" in request:
```

decides the tool.

That is not intelligent.

We want:

User Request



LLM



Understand Intent



Choose Correct Tool



Generate Tool Arguments



Application Executes Tool

This is exactly what we will build in:

Chapter 2 – Tool Calling

36. Agent Design Checklist

Before building an agent, ask:

1. what is the goal?
2. Does this actually need an agent?
3. which decisions require an LLM?
4. which tools are required?
5. what information belongs in state?
6. Does anything need persistent memory?
7. what are the stopping conditions?
8. what happens when a tool fails?
9. which actions need human approval?
10. what deterministic rules should exist?

This prevents unnecessary agent complexity.

37. Common Agent Design Mistakes

Mistake 1 – Using an Agent for Everything

If normal code solves the problem:

Use normal code.

Mistake 2 – Giving Too Many Tools

More tools can make selection harder.

Provide only relevant capabilities.

Mistake 3 – Poor Tool Descriptions

The model needs clear descriptions to choose correctly.

Mistake 4 – No State

Multi-step workflows require tracking intermediate information.

Mistake 5 – No Stopping Condition

Can cause unnecessary or infinite loops.

Mistake 6 – Unlimited Retries

Always control retry behavior.

Mistake 7 – Letting LLM Control Critical Rules

Use deterministic code for:

Permissions

Authentication

Financial limits

Security checks

Important business rules

Mistake 8 – Excessive Autonomy

Sensitive actions may require human approval.

38. Important Interview Questions

Q1. What is task decomposition?



Answer:

Task decomposition is the process of breaking a complex goal into smaller manageable tasks that can be executed individually to achieve the overall objective.

Q2. What is planning in an AI agent?



Answer:

Planning is the process through which an agent determines the sequence of tasks or actions required to achieve a goal and may modify that plan based on observations during execution.

Q3. Fixed workflow vs agentic workflow?



Answer:

A fixed workflow follows a predetermined sequence of steps, while an agentic workflow can dynamically choose its next action based on the current state and previous observations.

Q4. What is a planner-executor architecture?

Answer:

In a planner-executor architecture, the planner determines which tasks should be performed, while the executor carries out those tasks using available tools and returns observations.

Q5. What is the environment of an AI agent?

Answer:

The environment consists of external systems and information sources with which an agent can interact, such as APIs, databases, files, applications and knowledge bases.

Q6. What is a stopping condition?



Answer:

A stopping condition determines when an agent should terminate execution, such as when the goal is completed, the required information is obtained, a maximum step limit is reached or a timeout occurs.

Q7. Why are maximum step limits important?

Answer:

Maximum step limits prevent agents from entering infinite or unnecessarily long loops, reducing failures, latency and unexpected API costs.

Q8. What is reflection in an AI agent?



Answer:

Reflection is a pattern where an agent evaluates an intermediate result and decides whether it is sufficient or whether the action, plan or output should be revised and retried.

Q9. Does reflection guarantee correctness?

Answer:

No. Since an LLM can incorrectly evaluate its own output, reflection should be combined with validation, limits, deterministic checks or human review where appropriate.

Q10. Why combine deterministic code with agents? ★ ★ ★ ★ ★

Answer:

LLMs are useful for flexible language understanding and decision-making, while deterministic code provides predictable behavior for validation, permissions, limits, calculations and critical business rules.

Q11. Why are tool descriptions important?

Answer:

The LLM uses tool names, descriptions and schemas to determine when a tool is appropriate and which arguments should be provided.

Q12. What information can an agent use to make decisions?

Answer:

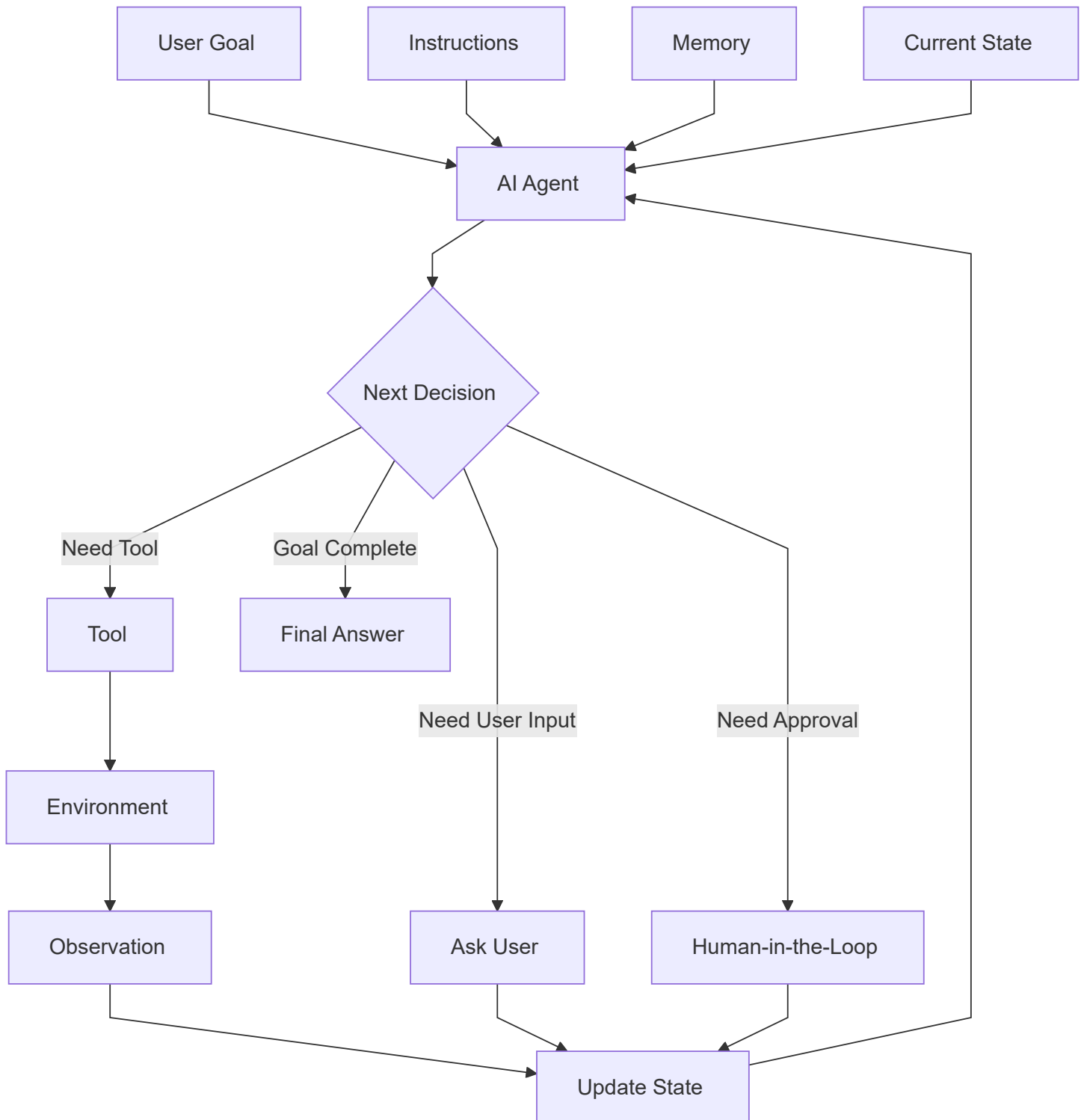
An agent may use the user goal, system instructions, available tool definitions, current state, previous observations and relevant memory to determine its next action.

Q13. What can an agent output besides a final answer?

Answer:

An agent may output a tool call, request additional user information, request human approval, continue the workflow or produce a final response.

39. Complete Chapter 1 Mental Model



Remember:

```
Goal
↓
Plan / Decide
↓
Action
↓
Observation
↓
Update State
↓
Decide Again
↓
Goal Complete
```

*A good agent does **not** mean:*

Give complete control to an LLM.

A better architecture is:

```
LLM
→ Flexible decisions

Tools
→ External capabilities

State
→ Track execution

Memory
→ Retain useful information

Normal Code
→ Rules and validation

Human
→ Approval where required
```

And before creating an agent, always ask:

Does this task really require dynamic decision-making?

If NO

→ Use a normal workflow.

If YES

→ Consider an agent.

The purpose of an agent is not to make a simple workflow complicated. It is to handle tasks where decisions, actions and future steps depend dynamically on the information discovered during execution.

End of Chapter I – Agentic AI Fundamentals
